

**ĐẠI HỌC PHENIKAA
TRƯỜNG KINH TẾ PHENIKAA
KHOA DU LỊCH – KHÁCH SẠN**



**HỌC PHẦN: LẬP TRÌNH ỨNG DỤNG WEB TRONG DU LỊCH 2
ĐỀ TÀI: “ Hệ thống quản lý và đặt thuê xe trực tuyến – CARENTO”**

Giảng viên: Phạm Đức Bắc

Lớp: Lập trình ứng dụng web trong du lịch 2 -N01

Thành Viên: Trần Thị Huyền - 22012989

Bùi Thị Quỳnh Anh - 22012417

Hà Nội – 2026

THÀNH VIÊN

STT	MSV	HỌ VÀ TÊN	VAI TRÒ & NHIỆM VỤ CHI TIẾT ĐỒ ÁN	TỶ LỆ ĐÓNG GÓP
1	22012989	Trần Thị Huyền	Trưởng nhóm, Phân công công việc,đôn đốc bảo đảm tiến độ. - Chịu trách nhiệm phân chia công việc, lập kế hoạch xây dựng WBS và quản lý tiến độ dự án. - Thiết kế kiến trúc tổng quan hệ thống MVC, xây dựng cơ sở dữ liệu quan hệ trên MySQL. -Lập trình toàn bộ phân hệ Khách hàng :Giao diện trang chủ, bộ lọc phân khúc phương tiện, trang chi tiết thông số xe và luồng đặt đơn thuê xe tính toán hóa đơn tạm tính thời gian thực. • Tích hợp hệ thống xác thực kép nâng cao: Gói bảo mật mã hóa tài khoản Laravel Breeze và Đăng nhập nhanh qua Google OAuth 2.0. • Lập trình module tiếp nhận thông tin liên hệ hỗ trợ và số điện thoại từ phía khách hàng • Chịu trách nhiệm cô lập biến môi trường, quản lý mã nguồn qua GitHub và trực tiếp triển khai toàn bộ ứng dụng lên máy chủ điện toán đám mây Railway (carento.up.railway.app). • Phối hợp biên soạn và hoàn thiện tài liệu báo cáo đồ án.	60%

2	22012417	Bùi Thị Quỳnh Anh	-Lập trình toàn bộ phân hệ Quản trị viên -Thiết kế giao diện trung tâm điều hành Dashboard tự động tổng hợp số liệu tính toán doanh thu thực tế và quản lý các chỉ số vận hành kinh doanh. -Xây dựng các module quản lý CRUD kho xe CRUD danh mục. -Xây dựng module quản lý danh sách hồ sơ thành viên khách hàng. -Xây dựng module điều phối, xử lý phê duyệt hoặc từ chối hủy đơn đặt xe đi kèm cơ chế khóa xe tự động chống trùng lịch biểu. -Xây dựng không gian làm việc tiếp nhận thông tin tin nhắn liên hệ hỗ trợ của khách hàng. -Chịu trách nhiệm lập kế hoạch, xây dựng ma trận kịch bản và thực hiện kiểm thử hộp đen cho toàn bộ các chức năng của hệ thống trên môi trường thật. -Chịu trách nhiệm chính trong việc biên soạn, định dạng cấu trúc nội dung và tổng hợp tài liệu báo cáo tổng kết đồ án môn học.	40%
---	----------	-------------------	--	-----

Mục Lục

THÀNH VIÊN.....	2
LỜI CẢM ƠN	7
DANH MỤC HÌNH ẢNH	8
DANH MỤC BẢNG	9
TÓM TẮT.....	10
GIỚI THIỆU	11
1. ĐẶT VẤN ĐỀ	11
2. LỊCH SỬ GIẢI QUYẾT VẤN ĐỀ	11
2.1. Những ưu điểm đã đạt được của các hệ thống hiện có.....	11
2.2. Những hạn chế còn tồn tại	12
2.3. Hướng giải quyết của đề tài CARENTO.....	12
3. MỤC TIÊU ĐỀ TÀI.....	13
3.1. Về mặt ứng dụng thực tiễn	13
3.2. Về mặt kỹ thuật và công nghệ	13
3.3. Kết quả mong đợi.....	14
4. PHẠM VI CỦA ĐỀ TÀI.....	14
4.1. Về mặt nghiệp vụ và lý thuyết	14
4.2. Về mặt Kỹ thuật và Công cụ	14
4.3. Về mặt Ứng dụng thực tiễn	15
5. PHƯƠNG PHÁP NGHIÊN CỨU.....	15
CHƯƠNG 1: MÔ TẢ BÀI TOÁN.....	17
1. MÔ TẢ TỔNG QUAN	17
1.1. Các chức năng của hệ thống.....	17
1.2. Nhóm người sử dụng.....	18
1.3. Môi trường vận hành	19
1.4. Ràng buộc về thực thi và thiết kế.....	19
2. CÁC YÊU CẦU GIAO TIẾP BÊN NGOÀI	19
2.1. Giao diện phần mềm (Software Interfaces)	19
2.2. Giao diện giao tiếp (Communication Interfaces)	20
2.3. Giao tiếp phần cứng (Hardware Interfaces).....	21
2.4. Giao tiếp truyền thông tin (Information Transmission Protocols)	21
3. CÁC CHỨC NĂNG CỦA HỆ THỐNG (ĐẶC TẢ CHI TIẾT RQ).....	22
3.1. RQ01 – Đăng ký tài khoản.....	22
3.2. RQ02 – Đăng nhập hệ thống	23
3.3. RQ03 – Đăng xuất tài khoản	25
3.4. RQ04 – Quản lý hồ sơ cá nhân (Profile)	26
3.5. RQ05 – Xem và lọc xe theo phân khúc	27
3.6. RQ06 – Xem chi tiết thông số xe.....	28

3.7. RQ07 – Đặt đơn thuê xe & Thanh toán (Mô phỏng)	28
3.8. RQ08 – Xem lịch sử đặt xe cá nhân	30
3.9. RQ09 – Gửi tin nhắn liên hệ hỗ trợ	30
3.10. RQ10 – Báo cáo thống kê tổng quan (Admin Dashboard)	31
3.11. RQ11 – Quản lý kho xe - CRUD Cars (Admin)	32
3.12. RQ12 – Quản lý danh sách thành viên khách hàng	34
3.13. RQ13 – Điều phối duyệt đơn đặt xe (Admin Bookings)	34
3.14. RQ14 – Quản lý tin nhắn liên hệ (Admin Messages)	35
4. CÁC YÊU CẦU PHI CHỨC NĂNG	36
4.1. Yêu cầu về hiệu năng thực thi (Performance Requirements)	36
4.2. Yêu cầu về độ tin cậy và xử lý ngoại lệ (Reliability & Exception Handling)	37
4.3. Yêu cầu về bảo mật an toàn thông tin (Security Requirements)	38
4.4. Yêu cầu về chất lượng và tính tiện dụng (Usability & Quality Requirements)	39
CHƯƠNG 2: THIẾT KẾ VÀ CÀI ĐẶT GIẢI PHÁP	41
1. KIẾN TRÚC HỆ THỐNG	41
1.1. Thiết kế kiến trúc tổng quan	41
1.2. Mô tả sự phân rã hệ thống (Component Decomposition)	42
1.2.1. Phân hệ Giao diện máy khách (Client-side Frontend)	42
1.2.2. Phân hệ Xử lý trung tâm (Server-side Backend)	42
1.2.3. Phân hệ Lưu trữ (Database Storage Layer)	43
2. THIẾT KẾ DỮ LIỆU	43
2.1. Mô hình dữ liệu quan hệ (Database Schema)	43
2.2. Từ điển dữ liệu chi tiết (Data Dictionary)	43
2.2.1. Bảng users (Lưu trữ thông tin tài khoản người dùng hệ thống)	43
2.2.2. Bảng categories (Quản lý danh mục phân khúc phương tiện)	44
2.2.3. Bảng cars (Quản lý thông tin chi tiết phương tiện ô tô)	45
2.2.4. Bảng bookings (Quản lý thông tin chi tiết đơn đặt thuê xe trực tuyến)	46
2.2.5. Bảng contacts (Quản lý tin nhắn liên hệ hỗ trợ từ khách hàng)	48
3. THIẾT KẾ CÁCH XỬ LÝ	49
3.1. Tiến trình Đăng ký tài khoản thành viên	49
3.2. Tiến trình Xác thực Đăng nhập (Mật khẩu & Google OAuth 2.0)	50
3.3. Tiến trình Đăng xuất tài khoản an toàn	51
3.4. Tiến trình Quản lý và cập nhật hồ sơ cá nhân (Profile)	52
3.5. Tiến trình Xem và lọc danh sách xe theo phân khúc danh mục	52
3.6. Tiến trình Xem chi tiết thông số kỹ thuật xe	53
3.7. Tiến trình Đặt đơn thuê xe & Xử lý hóa đơn tạm tính (Mô phỏng Checkout)	54
3.8. Tiến trình Xem lịch sử đơn đặt xe cá nhân của khách hàng	54
3.9. Tiến trình Tiếp nhận thông tin liên hệ hỗ trợ từ khách hàng (Client Contact)	55
3.10. Tiến trình Thống kê chỉ số báo cáo tại Trung tâm điều hành (Admin Dashboard)	56
3.11. Tiến trình Quản lý kho xe và Đổi trạng thái xe nhanh (Admin CRUD Cars)	56
3.12. Tiến trình Quản lý và kiểm soát danh sách thành viên khách hàng	57

3.13. Tiến trình Điều phối Phê duyệt / Hủy đơn đặt xe (Admin Bookings)	58
3.14. Tiến trình Tiếp nhận và xử lý Tin nhắn liên hệ của khách hàng (Admin Messages).....	59
4. KẾT QUẢ GIAO DIỆN WEB	59
4.1. GIAO DIỆN CHO MÁY TÍNH.....	60
4.1.1. Giao diện Trang đăng ký tài khoản thành viên mới	60
4.1.2. Giao diện Trang đăng nhập hệ thống (Xác thực kép).....	60
4.1.3. Giao diện Trang chủ và Bộ lọc phân khúc xe dành cho Khách hàng.....	61
4.1.4. Giao diện Trang hiển thị thông số kỹ thuật chi tiết xe.....	62
4.1.5. Giao diện Trang điền thông tin và Xác nhận đặt xe thuê (Checkout)	63
4.1.6. Giao diện Trang Lịch sử đơn đặt xe cá nhân của Khách hàng	64
4.1.7. Giao diện Trang Liên hệ hỗ trợ và Để lại số điện thoại tư vấn.....	64
4.1.8. Giao diện Trang trung tâm điều hành Admin Dashboard thống kê tài chính ..	65
4.1.9. Giao diện Trang Admin Quản lý kho xe và Đổi trạng thái phương tiện nhanh	65
4.1.10. Giao diện Trang Admin Quản lý danh sách thành viên khách hàng	66
4.1.11. Giao diện Trang Admin Điều phối Phê duyệt / Hủy đơn đặt xe	67
4.1.12. Giao diện Trang Admin Tiếp nhận quản lý tin nhắn liên hệ hỗ trợ.....	67
4.2. Giao diện điện thoại.....	68
4.2.1. Quy tắc chuyển đổi phân hệ Khách hàng (Client Mobile).....	68
4.2.2. Quy tắc chuyển đổi phân hệ Quản trị viên (Admin Mobile)	69
CHƯƠNG 3: KIỂM THỬ HỆ THỐNG	71
1. MỤC TIÊU KIỂM THỬ	71
2. PHƯƠNG PHÁP KIỂM THỬ	71
3. CÁC TRƯỜNG HỢP KIỂM THỬ	71
3.1. Trường hợp kiểm thử 1: Đăng ký tài khoản thành viên mới (RQ01).....	71
3.2. Trường hợp kiểm thử 2: Xác thực Đăng nhập (RQ02)	72
3.3. Trường hợp kiểm thử 3: Xem chi tiết thông số phương tiện xe (RQ06).....	74
3.4. Trường hợp kiểm thử 4: Đặt đơn thuê xe (RQ07).....	74
3.5. Trường hợp kiểm thử 5: Gửi tin nhắn liên hệ hỗ trợ (RQ09)	75
3.6. Trường hợp kiểm thử 6: Admin CRUD và Đổi trạng thái xe nhanh (RQ11).....	76
3.7. Trường hợp kiểm thử 7: Admin điều phối duyệt đơn và cập nhật doanh thu (RQ13).....	76
3.8. Trường hợp kiểm thử 8: Admin tiếp nhận quản lý tin nhắn liên hệ hỗ trợ (RQ14)	77
CHƯƠNG 4: KẾT LUẬN	78
4.1. KẾT QUẢ ĐẠT ĐƯỢC	78
4.2. NHỮNG HẠN CHẾ CÒN TỒN TẠI	78
4.3. HƯỚNG PHÁT TRIỂN TRONG TƯƠNG LAI.....	79

LỜI CẢM ƠN

Kính thưa thầy Phạm Đức Bắc,

Lời đầu tiên, chúng em xin phép được gửi lời tri ân và lòng biết ơn chân thành, sâu sắc nhất tới Thầy, với vai trò là người đã đồng hành và dẫn dắt chúng em trong môn học vô cùng quý giá – “Lập trình Web trong Du lịch”.

Sự tận tâm và nhiệt huyết của Thầy không chỉ là nền tảng tri thức vững chắc mà còn là nguồn cảm hứng, động lực to lớn và không ngừng nghỉ đối với toàn thể chúng em. Trong suốt quá trình học tập, Thầy đã dành trọn trái tim và sự uyên bác của mình để sẻ chia những kiến thức chuyên sâu về công nghệ và ứng dụng web trong bối cảnh đặc thù của ngành du lịch.

Chúng em đặc biệt ấn tượng và khắc cốt ghi tâm những giờ học đầy kỷ niệm mà Thầy đã dày công kiến tạo. Đó không chỉ là những buổi lên lớp đơn thuần mà còn là những buổi thực hành, phân tích case study về việc xây dựng các nền tảng đặt dịch vụ, website du lịch và cổng thông tin số. Sự kiên nhẫn, lòng nhiệt thành không mệt mỏi cùng phương pháp giảng dạy độc đáo của Thầy đã kiến tạo nên một môi trường học tập vô cùng tích cực, khích lệ tinh thần tự học, tự nghiên cứu và thôi thúc chúng em không ngừng nỗ lực, vươn lên trong học tập.

Những giá trị tinh thần và bài học quý báu mà Thầy đã truyền thụ không chỉ dừng lại ở kiến thức lập trình khô khan mà còn đã trở thành hành trang vững chắc, một sự đầu tư dài hạn cho tương lai nghề nghiệp của riêng chúng em. Những kiến thức, kinh nghiệm cùng nguồn cảm hứng về việc áp dụng công nghệ để giải quyết các bài toán du lịch sẽ luôn là kim chỉ nam soi sáng cho hành trình sự nghiệp sắp tới của chúng em.

Xin một lần nữa bày tỏ lòng biết ơn vô hạn đối với tất cả những điều tốt đẹp, những giá trị sâu sắc mà Thầy đã mang lại cho chúng em. Kính chúc Thầy dồi dào sức khỏe, hạnh phúc viên mãn và gặt hái thêm nhiều thành công rực rỡ trong sự nghiệp giáo dục.

Trân trọng và với lòng biết ơn sâu sắc nhất!

DANH MỤC HÌNH ẢNH

Hình 1: Giao diện Trang đăng ký tài khoản thành viên mới trên máy tính	60
Hình 2: Giao diện Trang đăng nhập tích hợp Google OAuth 2.0 trên máy tính	61
Hình 3: Giao diện Trang chủ và Bộ lọc danh mục phân khúc xe trên máy tính	61
Hình 4: Giao diện Trang thông số kỹ thuật chi tiết phương tiện trên máy tính 1	62
Hình 5: Giao diện Trang thông số kỹ thuật chi tiết phương tiện trên máy tính 2	62
Hình 6: Giao diện Form đặt xe và Hóa đơn tạm tính thời gian thực trên máy tính 1	63
Hình 7: Giao diện Form đặt xe và Hóa đơn tạm tính thời gian thực trên máy tính 2	63
Hình 8: Giao diện Bảng quản lý lịch sử đơn đặt xe của Khách hàng trên máy tính	64
Hình 9: Giao diện Form gửi thông tin liên hệ hỗ trợ tư vấn trên máy tính	64
Hình 10: Giao diện Trung tâm điều hành Admin Dashboard thống kê trên máy tính	65
Hình 11: Giao diện Bảng quản lý kho xe và nút đổi trạng thái nhanh trên máy tính	65
Hình 12: Giao diện Trang Admin quản lý hồ sơ danh sách thành viên trên máy tính 1	66
Hình 13: Giao diện Trang Admin quản lý hồ sơ danh sách thành viên trên máy tính 2	66
Hình 14: Giao diện Trang Admin điều phối phê duyệt đơn đặt xe trên máy tính	67
Hình 15: Giao diện Không gian làm việc quản lý tin nhắn liên hệ của Admin trên máy tính	67

DANH MỤC BẢNG

Bảng 1.1: Tổng hợp phân hệ chức năng thực tế của hệ thống CARENTO
Bảng 2.1: Cấu trúc dữ liệu bảng người dùng (users)
Bảng 2.2: Cấu trúc dữ liệu bảng danh mục phân khúc (categories)
Bảng 2.3: Cấu trúc dữ liệu bảng chi tiết phương tiện (cars)
Bảng 2.4: Cấu trúc dữ liệu bảng đơn đặt thuê xe (bookings)
Bảng 2.5: Cấu trúc dữ liệu bảng tin nhắn liên hệ hỗ trợ (contacts)
Bảng 3.1: Kịch bản kiểm thử chức năng Đăng ký tài khoản (RQ01)
Bảng 3.2: Kịch bản kiểm thử chức năng Xác thực Đăng nhập (RQ02)
Bảng 3.3: Kịch bản kiểm thử chức năng Xem chi tiết thông số xe (RQ06)
Bảng 3.4: Kịch bản kiểm thử chức năng Đặt đơn thuê xe tự lái (RQ07)
Bảng 3.5: Kịch bản kiểm thử chức năng Gửi tin nhắn liên hệ hỗ trợ (RQ09)
Bảng 3.6: Kịch bản kiểm thử chức năng Admin CRUD và Đổi trạng thái xe nhanh (RQ11)
Bảng 3.7: Kịch bản kiểm thử chức năng Admin duyệt đơn và cập nhật doanh thu (RQ13)
Bảng 3.8: Kịch bản kiểm thử chức năng Admin quản lý tin nhắn liên hệ hỗ trợ (RQ14).....

TÓM TẮT

Trong xu thế chuyển đổi số toàn diện đang diễn ra mạnh mẽ, việc tối ưu hóa quy trình vận hành và nâng cao trải nghiệm người dùng thông qua các nền tảng thương mại điện tử trực tuyến đã trở thành yêu cầu bắt buộc đối với mọi ngành dịch vụ, bao gồm cả lĩnh vực cho thuê phương tiện giao thông. Đề án “Xây dựng hệ thống quản lý và đặt thuê xe trực tuyến – CARENTO” được thực hiện nhằm mục đích thay thế phương thức vận hành thủ công truyền thống bằng một giải pháp công nghệ hiện đại, tự động hóa và có độ bảo mật cao.

Về mặt công nghệ, ứng dụng được phát triển dựa trên Laravel Framework – một cấu trúc backend PHP mạnh mẽ và chuẩn mực theo mô hình MVC, kết hợp với bộ công cụ Laravel Breeze và Laravel Socialite để thiết lập hệ thống quản lý phiên và xác thực người dùng kép (qua tài khoản hệ thống và qua Google OAuth 2.0). Phía giao diện (Frontend) được thiết kế đồng bộ bằng Tailwind CSS, tối ưu hóa asset thông qua trình biên dịch tốc độ cao Vite (Node.js) nhằm đảm bảo giao diện thích ứng tốt trên cả máy tính (Desktop) lẫn điện thoại di động (Mobile). Cơ sở dữ liệu của hệ thống sử dụng MySQL, được cấu hình chặt chẽ với các ràng buộc khóa ngoại nhằm đảm bảo tính toàn vẹn dữ liệu. Toàn bộ ứng dụng và cơ sở dữ liệu đã được triển khai (Deploy) thực tế trên nền tảng đám mây Railway tại địa chỉ domain carento.up.railway.app.

Kết quả đạt được là một website thương mại điện tử hoàn chỉnh cho phép khách hàng thực hiện quy trình khép kín từ tìm kiếm xe, lọc theo danh mục phân khúc, xem chi tiết thông số và tiến hành đặt xe trực tuyến. Song song đó, hệ thống cung cấp cho nhà quản trị một trung tâm điều hành (Admin Dashboard) mạnh mẽ để kiểm soát danh mục xe, trạng thái xe, phê duyệt đơn đặt và tự động cập nhật báo cáo tổng doanh thu theo thời gian thực, đặt nền móng vững chắc cho một sản phẩm thương mại thực thụ.

GIỚI THIỆU

1. ĐẶT VẤN ĐỀ

Cùng với sự tăng trưởng ổn định của nền kinh tế và thu nhập bình quân đầu người tại Việt Nam, nhu cầu sử dụng ô tô làm phương tiện di chuyển trong các dịp du lịch, công tác, hay cưới hỏi của người dân đang chứng kiến mức tăng trưởng vượt bậc. Tuy nhiên, việc sở hữu và duy trì một chiếc ô tô cá nhân đòi hỏi chi phí đầu tư và bảo dưỡng rất lớn, khiến dịch vụ thuê xe tự lái trở thành một giải pháp kinh tế và linh hoạt được đại đa số người dân ưu tiên lựa chọn.

Mặc dù có nhu cầu thị trường rất lớn, quy trình cho thuê xe truyền thống hiện nay tại nhiều đơn vị nhỏ lẻ vẫn đang đối mặt với nhiều bất cập lớn:

- +) Đối với khách hàng: Thông tin về các dòng xe, giá cả, thông số kỹ thuật và lịch xe trống thường không công khai minh bạch. Người thuê phải liên hệ qua nhiều khâu trung gian, thủ tục giấy tờ giữ xe phức tạp và dễ gặp rủi ro bị tăng giá vào các mùa cao điểm.
- +) Đối với đơn vị quản lý: Việc điều phối lịch trình xe, theo dõi trạng thái xe (đang trống, đang được thuê, hay đang bảo dưỡng) chủ yếu ghi chép thủ công bằng sổ sách hoặc file Excel thô sơ. Quy trình này rất dễ dẫn đến sai sót chồng chéo lịch xe, thất thoát dữ liệu lịch sử khách hàng và khó khăn trong việc thống kê doanh thu chính xác.

Đứng trước bài toán cấp thiết về tối ưu hóa quy trình vận hành, việc ứng dụng công nghệ thông tin để xây dựng một nền tảng quản lý tập trung là giải pháp tất yếu. Hệ thống CARENTO ra đời nhằm mục đích giải quyết triệt để các hạn chế trên, mang lại một kênh tương tác minh bạch, nhanh chóng cho người dùng và một công cụ quản trị khoa học, an toàn cho doanh nghiệp.

2. LỊCH SỬ GIẢI QUYẾT VẤN ĐỀ

2.1. Những ưu điểm đã đạt được của các hệ thống hiện có

Trên thị trường hiện nay đã xuất hiện một số ứng dụng và website công nghệ hỗ

trợ thuê xe quy mô lớn (như Mito, Zoomcar, hoặc các cổng thông tin của các hãng xe lớn). Qua quá trình khảo sát, các hệ thống này đã đạt được những thành tựu cơ bản bao gồm:

- +) Xây dựng được bộ lọc tìm kiếm phương tiện theo tỉnh thành và thời gian nhận/trả xe khá trực quan.
- +) Số hóa được một phần thông tin hình ảnh xe và niêm yết mức giá thuê theo ngày rõ ràng cho người dùng đối chiếu.
- +) Xây dựng được hệ thống lưu trữ thông tin lịch sử giao dịch của tài khoản ở mức độ cơ bản trên môi trường số.

2.2. Những hạn chế còn tồn tại

Tuy nhiên, khi xét ở quy mô các đề án công nghệ nội địa hoặc các hệ thống vận hành của các nhà xe tầm trung, các hạn chế lớn vẫn đang hiện hữu:

- +) Công nghệ lạc hậu, vỡ giao diện: Nhiều website cho thuê xe hiện tại vẫn sử dụng mã nguồn PHP thuần cũ kỹ hoặc mã nguồn WordPress nặng nề, không tối ưu được tốc độ tải trang. Đặc biệt, việc thiếu công nghệ Responsive khiến giao diện bị vỡ nát khi khách hàng truy cập bằng thiết bị di động.
- +) Rủi ro bảo mật và trải nghiệm kém khi đăng ký: Các hệ thống tự code cũ thường lưu trữ mật khẩu dưới dạng text thô hoặc md5 sơ sài, đồng thời bắt buộc khách hàng điền các form đăng ký rất dài dòng, gây mất thời gian và làm giảm tỷ lệ chuyển đổi người dùng mới.
- +) Quy trình vận hành chưa đồng bộ lên mây: Phần lớn các dự án sinh viên hoặc hệ thống thử nghiệm chỉ dừng lại ở việc vận hành cục bộ trên môi trường máy tính cá nhân (Localhost), dẫn đến việc không kiểm chứng được các lỗi phát sinh thực tế liên quan đến giao thức bảo mật (HTTPS) hay việc phân tách biến môi trường khi đưa ứng dụng lên internet thật.

2.3. Hướng giải quyết của đề tài CARENTO

Nhận thấy rõ các điểm nghẽn kỹ thuật và nghiệp vụ trên, đề tài CARENTO được

thiết kế và triển khai với các giải pháp công nghệ đột phá nhằm khắc phục hoàn toàn các nhược điểm cũ:

- +) Xây dựng trên Framework hiện đại: Thay vì PHP thuần, hệ thống sử dụng Laravel chuẩn kiến trúc MVC kết hợp cùng Tailwind CSS và Vite. Sự kết hợp này mang lại tốc độ phản hồi trang web cực nhanh (chỉ tính bằng mili giây) và cam kết giao diện co giãn hiển thị chuẩn mực trên mọi màn hình từ Mac, PC cho đến iPhone, Android.
- +) Tích hợp xác thực kép bảo mật cao: Hệ thống ứng dụng Laravel Breeze kết hợp Socialite, cho phép khách hàng đăng nhập nhanh bằng Google OAuth 2.0 chỉ với 1 cú click chuột, vừa nâng cao bảo mật vừa đơn giản hóa trải nghiệm người dùng.
- +) Triển khai thực tế trên môi trường Điện toán đám mây (Cloud): Ứng dụng được deploy trực tiếp lên nền tảng Railway. Toàn bộ các thông tin nhạy cảm của hệ thống như mật khẩu database hay Client Secret của Google được mã hóa an toàn thông qua tính năng Variables của Railway, tách biệt hoàn toàn khỏi mã nguồn trên GitHub, tuân thủ nghiêm ngặt tiêu chuẩn bảo mật của các lập trình viên chuyên nghiệp.

3. MỤC TIÊU ĐỀ TÀI

3.1. Về mặt ứng dụng thực tiễn

- Xây dựng một website thương mại điện tử hoàn chỉnh cho dịch vụ thuê xe tự lái, vận hành trơn tru trong môi trường internet thực tế.
- Tự động hóa khép kín: Chuyển đổi toàn bộ quy trình đặt xe, tính toán tiền thuê, áp dụng mã giảm giá và theo dõi trạng thái đơn hàng sang cơ chế tự động 24/7.
- Quản lý khoa học: Cung cấp cho Admin một bảng điều khiển Dashboard trung tâm để quản lý vòng đời của xe (từ lúc thêm mới, cập nhật giá, trạng thái sẵn sàng cho đến khi ẵm xe) và duyệt đơn đặt xe một cách chính xác.

3.2. Về mặt kỹ thuật và công nghệ

- Làm chủ hoàn toàn kiến trúc phần mềm MVC (Model - View - Controller) chuẩn

mức của framework Laravel để mã nguồn rõ ràng, dễ bảo trì và dễ mở rộng.

- Ứng dụng thành thạo quy trình đóng gói và biên dịch tài nguyên Front-end (Compile Assets) thông qua môi trường Node.js và Vite config, giải quyết triệt để bài toán rút phong chữ tiếng Việt (bằng cách ép font Inter toàn hệ thống) và lỗi Manifest trên server Production.

- Tổ chức cấu trúc cơ sở dữ liệu quan hệ MySQL chuẩn hóa, xử lý mượt mà các truy vấn kết hợp nhiều bảng (Database Queries) để tính toán doanh thu và tổng số đơn hàng cần xử lý.

3.3. Kết quả mong đợi

- Một hệ thống website mang tên miền riêng hoạt động thực tế với giao diện chuyên nghiệp, thẩm mỹ cao, thu hút người dùng.

- Hệ thống không mắc các sai sót về bảo mật cơ bản, xử lý ngoại lệ tốt, không văng lỗi 500 hệ thống khi người dùng thao tác sai.

- Báo cáo thống kê của Admin hiển thị biểu đồ và số liệu chính xác tuyệt đối ngay sau khi một đơn đặt xe được duyệt hoặc hủy.

4. PHẠM VI CỦA ĐỀ TÀI

4.1. Về mặt nghiệp vụ và lý thuyết

- Nghiệp vụ: Tập trung sâu vào luồng xử lý thương mại điện tử đặc thù của mảng cho thuê xe: Quy trình kiểm tra danh mục xe, phân loại xe theo hãng; Quy trình quản lý vòng đời trạng thái của xe (Available / Rented / Maintenance); Quy trình đặt chỗ tạm tính theo số ngày thuê thực tế.

- Lý thuyết: Nghiên cứu sâu về cơ chế hoạt động của middleware trong Laravel (nhận diện quyền khách hàng và quyền Admin), cơ chế đồng bộ hóa dữ liệu thông qua Eloquent ORM và cách thức cấu hình Route Callback an toàn với Google Cloud API.

4.2. Về mặt Kỹ thuật và Công cụ

- Hệ thống được phát triển và vận hành dựa trên các công cụ tiêu chuẩn:

- +) Môi trường phát triển dưới máy (Local): Sử dụng Visual Studio Code làm IDE chính; Quản lý các gói thư viện PHP backend bằng Composer;

Quản lý tài nguyên Front-end bằng NPM và Node.js.

+) Môi trường quản trị và kiểm thử CSDL: Sử dụng phần mềm TablePlus để kết nối, truy vấn dữ liệu và thực hiện các thao tác Import cấu trúc bảng từ các tệp dữ liệu mẫu.

+) Môi trường triển khai thực tế (Cloud): Nền tảng PaaS Railway kết hợp kho lưu trữ mã nguồn đám mây GitHub Repository.

+) Ngôn ngữ sử dụng: PHP (Backend), HTML5/Blade Template Engine, CSS3/Tailwind, JavaScript (Frontend).

4.3. Về mặt Ứng dụng thực tiễn

- Ứng dụng giới hạn các phân hệ chức năng cụ thể bao gồm:

+) Giao diện khách hàng: Đăng ký tài khoản, Đăng nhập (Mật khẩu / Google), Xem danh sách xe, Xem chi tiết thông số xe, Đặt xe, Quản lý lịch sử và trạng thái đơn thuê xe cá nhân.

+) Giao diện quản trị (Admin Panel): Dashboard tổng quan thống kê các chỉ số kinh doanh; Quản lý CRUD Danh mục phân khúc xe; Quản lý CRUD Thông tin xe; Quản lý danh sách khách hàng; Module điều phối duyệt/hủy đơn đặt xe thời gian thực.

+) Dữ liệu kiểm thử: Hệ thống sử dụng tập dữ liệu mẫu chuẩn về các dòng xe ô tô phổ biến tại thị trường Việt Nam phục vụ cho việc demo và kiểm thử hộp đen.

5. PHƯƠNG PHÁP NGHIÊN CỨU

- Phương pháp nghiên cứu lý thuyết: Đọc và phân tích các tài liệu kỹ thuật chính thống (Documentation) của Laravel Framework, Tailwind CSS và các tài liệu hướng dẫn tích hợp Google OAuth 2.0 API để xây dựng sơ đồ chức năng và sơ đồ luồng dữ liệu chuẩn xác.

- Phương pháp thực nghiệm nghiên cứu phát triển: Tiến hành lập trình thực tế, xây dựng mã nguồn theo từng giai đoạn (phát triển cục bộ dưới máy -> đóng gói assets qua Vite -> cấu hình bảo mật .env -> đẩy lên GitHub -> cấu hình Variables và deploy lên mây

Railway).

- Phương pháp kiểm thử thực tế: Áp dụng phương pháp kiểm thử hộp đen (Black-box testing) đóng vai trò người dùng cuối và nhà quản trị để tiến hành đăng ký, đăng nhập, đặt đơn và duyệt đơn trực tiếp trên đường link thật nhằm phát hiện, khắc phục triệt để các lỗi xung đột hệ thống.

CHƯƠNG 1: MÔ TẢ BÀI TOÁN

1. MÔ TẢ TỔNG QUAN

1.1. Các chức năng của hệ thống

Hệ thống quản lý và đặt thuê xe trực tuyến CARENTO được thiết kế nhằm phục vụ hai nhóm đối tượng mục tiêu chính là Khách hàng có nhu cầu thuê phương tiện và Ban quản trị nhà xe (Admin). Các phân hệ chức năng tổng quan được mô tả chi tiết trong bảng dưới đây:

STT	Mã chức năng	Tên chức năng	Phân hệ tương tác	Mô tả tóm tắt nghiệp vụ hệ thống
1	RQ01	Đăng ký tài khoản	Khách hàng vắng lai	Khởi tạo tài khoản thành viên mới trên hệ thống.
2	RQ02	Đăng nhập hệ thống	Tất cả người dùng	Xác thực bằng tài khoản mã hóa hoặc qua Google OAuth 2.0.
3	RQ03	Đăng xuất tài khoản	Tất cả người dùng	Xóa bỏ cookie phiên, kết thúc phiên làm việc an toàn.
4	RQ04	Quản lý hồ sơ cá nhân	Tất cả người dùng	Thay đổi thông tin cá nhân, cập nhật hoặc xóa tài khoản.
5	RQ05	Xem và lọc xe theo phân khúc	Khách hàng thành viên	Hiển thị danh sách xe, lọc nhanh xe theo danh mục phân khúc.
6	RQ06	Xem chi tiết thông số xe	Khách hàng thành viên	Xem chi tiết hình ảnh, hộp số, nhiên liệu và đơn giá thuê theo ngày.
7	RQ07	Đặt đơn thuê xe & Thanh toán	Khách hàng thành viên	Nhập ngày nhận/trả, tính toán chi phí và thực hiện thanh toán mô phỏng.
8	RQ08	Xem lịch sử đặt xe cá nhân	Khách hàng thành viên	Theo dõi danh sách và trạng thái xử lý các đơn đặt

				xe của cá nhân.
9	RQ09	Gửi tin nhắn liên hệ hỗ trợ	Khách hàng thành viên	Điền form yêu cầu gọi lại để nhân viên tư vấn trực tiếp trực tiếp.
10	RQ10	Báo cáo thống kê tổng quan	Quản trị viên (Admin)	Dashboard thống kê tổng doanh thu, lượng đơn hàng và số yêu cầu.
11	RQ11	Quản lý kho xe (CRUD)	Quản trị viên (Admin)	Thêm xe, sửa thông số, xóa xe hoặc đổi trạng thái xe nhanh trên Grid.
12	RQ12	Quản lý danh sách thành viên	Quản trị viên (Admin)	Xem danh sách khách hàng, theo dõi tổng số đơn đặt của từng người.
13	RQ13	Điều phối duyệt đơn đặt xe	Quản trị viên (Admin)	Phê duyệt (Approve) hoặc Hủy (Cancel) các đơn đặt xe chờ xử lý.
14	RQ14	Quản lý tin nhắn liên hệ	Quản trị viên (Admin)	Tiếp nhận thông tin yêu cầu tư vấn để gọi điện làm việc trực tiếp.

Bảng 1. Các chức năng của hệ thống

1.2. Nhóm người sử dụng

Hệ thống CARENTO phân định quyền hạn rõ ràng cho hai nhóm đối tượng thông qua mã kiểm tra Middleware:

- Khách hàng (Client): Bao gồm khách vãng lai và khách thành viên đã đăng nhập. Nhóm này sử dụng giao diện Front-end công khai để xem thông tin sản phẩm xe, gửi đơn đặt xe trực tuyến hoặc gửi form liên hệ khi cần nhân viên tư vấn sâu hơn.
- Quản trị viên (Admin): Đội ngũ điều hành nhà xe. Nhóm này có toàn quyền truy cập phân hệ Back-end (/admin/*) để quản lý tài sản xe, xử lý đơn đặt hàng và tiếp nhận danh sách tin nhắn của khách để phân phối nhân viên gọi điện trực tiếp làm việc.

1.3. Môi trường vận hành

- Môi trường Server: Mã nguồn PHP v8.4 chạy trên nền Laravel Framework v13.9, kết hợp hệ quản trị cơ sở dữ liệu MySQL v8.0. Toàn bộ cấu trúc được triển khai đám mây hóa trên máy chủ ảo PaaS Railway.

- Môi trường Client: Tương thích tốt trên cả Desktop và Mobile nhờ công nghệ đóng gói tối ưu của Vite Compiler kết hợp với CSS đa nền tảng của Tailwind CSS.

- Quản lý phiên bản và mã nguồn (Version Control): Sử dụng Git và GitHub để quản lý mã nguồn tập trung, hỗ trợ làm việc nhóm hiệu quả, theo dõi lịch sử thay đổi và xử lý xung đột mã nguồn (Merge Conflict), tạo tiền đề cho quy trình tích hợp liên tục (CI/CD) sau này, đồng thời tuân thủ quy trình phát triển phần mềm chuyên nghiệp.

1.4. Ràng buộc về thực thi và thiết kế

- Hệ thống đặt ra ràng buộc chặt chẽ về mặt thiết kế giao diện: Các form biểu mẫu nhập liệu (Form đặt xe, Form liên hệ) phải ngắn gọn, tinh giản tối đa các bước trung gian để khách hàng dễ dàng để lại thông tin mà không gặp rào cản phức tạp.

2. CÁC YÊU CẦU GIAO TIẾP BÊN NGOÀI

2.1. Giao diện phần mềm (Software Interfaces)

- Phân hệ giao diện phần mềm của hệ thống CARENTO định nghĩa cách thức ứng dụng tương tác với hệ điều hành máy chủ, môi trường thực thi backend, hệ quản trị cơ sở dữ liệu và các trình duyệt phía máy khách:

- Tương tác với Môi trường thực thi và Web Server: Ứng dụng tương tác chặt chẽ với trình thông dịch PHP v8.4 và máy chủ FrankenPHP/Apache được cài đặt trong môi trường Linux Container của nền tảng PaaS Railway. Hệ thống sử dụng công cụ quản lý thư viện Composer để kết nối và điều phối các thành phần mã nguồn cốt lõi của Laravel Framework v13.9.

- Cơ chế xử lý giao diện động (Presentation Layer): Hệ thống ứng dụng công cụ cấu trúc giao diện Blade Template Engine để xây dựng các trang hiển thị động. Cơ chế này cho phép phân tách hoàn toàn giữa logic nghiệp vụ (Business Logic) nằm ở các lớp điều khiển (ContactController, MessageController, CustomerController) và tầng hiển thị

giao diện (UI).

- Giao tiếp và Đóng gói Tài nguyên Đồ họa: Phía giao diện máy khách sử dụng Tailwind CSS Framework để định cấu hình phong cách thiết kế phẳng. Để tối ưu hóa hiệu năng tải tệp, hệ thống giao tiếp với trình biên dịch tài nguyên Vite Compiler (Node.js) thông qua tệp cấu hình vite.config.js. Trình biên dịch này băm nhỏ và ánh xạ trực tiếp các trang chức năng đặc thù với các tệp định dạng giao diện tĩnh tương ứng như:

- Trình quản lý danh sách khách hàng (/admin/customers) liên kết dữ liệu định dạng cấu trúc đồ họa với tệp customers.css.

- Trình tiếp nhận và điều phối tin nhắn liên hệ (/admin/messages) liên kết dữ liệu đồ họa hiển thị trực quan với tệp messages.css.

- Giao tiếp Cơ sở dữ liệu: Hệ thống giao tiếp với hệ quản trị cơ sở dữ liệu MySQL v8.0 thông qua lớp trừu tượng dữ liệu Eloquent ORM. Lớp này chuyển đổi các thao tác lập trình hướng đối tượng từ PHP thành các câu lệnh truy vấn cấu trúc SQL thuần túy một cách an toàn và bảo mật.

2.2. Giao diện giao tiếp (Communication Interfaces)

- Giao diện giao tiếp xác định các giao thức truyền thông mạng và cơ chế bảo mật dòng dữ liệu được luân chuyển giữa máy khách (Client Browser) và máy chủ đám mây:

- Giao thức mạng cốt lõi: Hệ thống bắt buộc thiết lập giao thức truyền tải siêu văn bản bảo mật HTTPS (Hypertext Transfer Protocol Secure) trên nền tảng mã hóa an toàn lớp truyền tải TLS/SSL. Mọi luồng dữ liệu thô từ máy khách khi gửi lên máy chủ Railway đều được mã hóa bất đối xứng, ngăn chặn triệt để các cuộc tấn công nghe lén hoặc giả mạo gói tin trên đường truyền mạng (Man-in-the-Middle attacks).

Bảo mật dữ liệu định danh khách hàng: Các dữ liệu riêng tư nhạy cảm của người dùng bao gồm: Họ và tên, Email, Số điện thoại liên lạc khẩn cấp và Nội dung tin nhắn để lại yêu cầu gọi lại tư vấn, bắt buộc phải truyền tải qua kênh mã hóa này để bảo vệ quyền riêng tư tuyệt đối cho khách hàng.

- Giao tiếp API Xác thực bên thứ ba: Hệ thống tích hợp giao thức xác thực mở chuẩn công nghiệp Google OAuth 2.0 thông qua gói thư viện Socialite. Quá trình trao đổi mã bảo mật giữa ứng dụng và máy chủ Google API được thực hiện an toàn qua đường

dẫn phản hồi được mã hóa hoàn toàn.

2.3. Giao tiếp phần cứng (Hardware Interfaces)

- Giao tiếp phần cứng mô tả cách thức phần mềm ứng dụng tương tác và khai thác tài nguyên từ các thiết bị phần cứng vật lý ở cả hai đầu hệ thống:

+) Phía Máy chủ (Server-side): Ứng dụng được vận hành trong các gói tài nguyên phần cứng ảo hóa (Virtual Hardware Resources) do hạ tầng đám mây PaaS Railway cung cấp. Hệ thống thiết lập giao tiếp phần cứng trừu tượng tối thiểu bao gồm: 1 vCPU xử lý thuật toán luồng, 1GB dung lượng bộ nhớ truy cập ngẫu nhiên (RAM) để duy trì các phiên làm việc (Sessions) đồng thời, và dung lượng ổ cứng trạng thái rắn (SSD) để phục vụ cho các tiến trình đọc/ghi tệp tin nhật ký hệ thống (System Logs) cũng như lưu trữ hình ảnh phương tiện xe ô tô.

+) Phía Máy khách (Client-side): Ứng dụng tiếp nhận và xử lý các tín hiệu điều khiển vào/ra (I/O) tiêu chuẩn của thiết bị phần cứng người dùng bao gồm: Thiết bị nhập liệu (Bàn phím, Chuột máy tính hoặc Màn hình cảm ứng đa điểm của thiết bị di động) và Thiết bị xuất dữ liệu (Màn hình hiển thị đồ họa). Giao diện phần mềm tự động tính toán lại kích thước ma trận pixel của phần cứng để co giãn giao diện hiển thị phù hợp thông qua cơ chế Responsive Design của Tailwind CSS.

2.4. Giao tiếp truyền thông tin (Information Transmission Protocols)

- Mục này định nghĩa cơ chế đóng gói dữ liệu và các phương thức truyền tải trạng thái tài nguyên trong mô hình kiến trúc MVC của Laravel:

+) Phương thức truyền tải yêu cầu (HTTP Request Methods): Hệ thống chuẩn hóa luồng di chuyển của dữ liệu thông qua 3 phương thức truyền tin cốt lõi:

+) Phương thức GET: Sử dụng để yêu cầu máy chủ truy vấn dữ liệu an toàn và trả về giao diện hiển thị mà không làm thay đổi trạng thái hệ thống (ví dụ: Truy cập trang liên hệ /lien-he, xem danh sách khách hàng

/admin/customers, hoặc duyệt tin nhắn /admin/messages).

+) Phương thức POST: Sử dụng để đóng gói dữ liệu từ các form biểu mẫu của người dùng và gửi lên máy chủ để khởi tạo một tài nguyên mới (ví dụ: Khách hàng gửi thông tin liên hệ, để lại số điện thoại qua route /contact/submit, hoặc gửi đơn đặt xe trực tuyến qua route /dat-xe).

+) Phương thức PATCH: Sử dụng để gửi các yêu cầu cập nhật một phần dữ liệu hoặc thay đổi trạng thái nhanh của tài nguyên trong CSDL mà không cần tải lại toàn bộ trang web (ví dụ: Admin thay đổi nhanh trạng thái vận hành của phương tiện xe qua route /admin/cars/{id}/status hoặc điều phối cập nhật vòng đời duyệt/hủy đơn hàng qua route /admin/bookings/{id}/status).

+) Định dạng đóng gói dữ liệu: Dữ liệu truyền đi từ form biểu mẫu của Client được mã hóa theo chuẩn application/x-www-form-urlencoded. Đối với luồng dữ liệu trả về từ API Google, thông tin được đóng gói dưới dạng các chuỗi đối tượng JSON (JavaScript Object Notation) có cấu trúc tường minh để Controller dễ dàng phân tích và xử lý.

+) Quản lý trạng thái thông tin (State Management): Thông tin trạng thái đăng nhập và xác thực của người dùng được duy trì liên tục giữa Client và Server thông qua các tệp tin Cookie mã hóa bảo mật bao gồm token chống giả mạo chéo trường XSRF-TOKEN và chuỗi định danh phiên làm việc laravel_session.

3. CÁC CHỨC NĂNG CỦA HỆ THỐNG (ĐẶC TẢ CHI TIẾT RQ)

3.1. RQ01 – Đăng ký tài khoản

- **Tên chức năng:** Đăng ký tài khoản thành viên mới.
- **Mô tả:** Cho phép khách hàng vắng lai tạo tài khoản định danh trên hệ thống để sử dụng các dịch vụ đặt xe và gửi liên hệ.
- **Tác nhân (Actor):** Khách hàng vắng lai.
- **Tiền điều kiện (Pre-conditions):** Người dùng truy cập hệ thống công khai và chưa đăng nhập tài khoản.

- **Hậu điều kiện (Post-conditions):** Một bản ghi người dùng mới được khởi tạo thành công trong bảng users với trường role = 'user'.

- **Luồng sự kiện chính (Basic Flow):**

1. Người dùng nhấn nút "Đăng ký" trên thanh điều hướng trang chủ.
2. Hệ thống chuyển hướng, hiển thị Form đăng ký tài khoản với các trường: Họ và tên, Địa chỉ Email, Mật khẩu, Xác nhận mật khẩu.
3. Người dùng tiến hành nhập đầy đủ thông tin cá nhân vào form và nhấn chọn nút "Tạo tài khoản".
4. Hệ thống tiếp nhận dữ liệu, kích hoạt lớp kiểm thử dữ liệu đầu vào (Validation).
5. Dữ liệu hoàn toàn hợp lệ, hệ thống tự động băm mã hóa mật khẩu thông qua thuật toán Bcrypt.
6. Hệ thống thực hiện câu lệnh INSERT để lưu tài khoản vào cơ sở dữ liệu MySQL, thiết lập quyền hạn mặc định là khách hàng.
7. Hệ thống khởi tạo phiên làm việc (Session), thông báo đăng ký thành công và tự động điều hướng người dùng về giao diện /dashboard.

- **Luồng sự kiện ngoại lệ (Exception Flow):**

1. Trường hợp Email đã tồn tại hoặc sai định dạng: Tại bước 4, nếu hệ thống phát hiện Email đã có tài khoản sử dụng hoặc nhập thiếu ký tự @, hệ thống sẽ chặn tiến trình, giữ lại dữ liệu cũ và xuất thông báo lỗi: "Email này đã được sử dụng trên hệ thống" hoặc "Định dạng Email không hợp lệ".
2. Trường hợp Mật khẩu không trùng khớp hoặc quá ngắn: Tại bước 4, nếu ô xác nhận mật khẩu không khớp hoặc độ dài mật khẩu dưới 8 ký tự, hệ thống hiển thị lỗi: "Mật khẩu xác nhận không trùng khớp" hoặc "Mật khẩu phải chứa ít nhất 8 ký tự".

3.2. RQ02 – Đăng nhập hệ thống

- **Tên chức năng:** Xác thực đăng nhập (Hệ thống truyền thống & Google OAuth 2.0).

- **Mô tả:** Xác định danh tính người dùng truy cập để cấp quyền điều phối giao diện tương ứng (Client hoặc Admin).
- **Tác nhân (Actor):** Khách hàng / Quản trị viên (Admin).
- **Tiền điều kiện (Pre-conditions):** Người dùng đã sở hữu tài khoản thành viên hoặc tài khoản Google.
- **Hậu điều kiện (Post-conditions):** Phiên làm việc (Session) bảo mật được thiết lập, mã hóa cookie trên trình duyệt máy khách.
- **Luồng sự kiện chính (Basic Flow):**
 - Luồng đăng nhập bằng tài khoản hệ thống:
 1. Người dùng truy cập trang Đăng nhập, nhập Email và Mật khẩu vào form rồi ấn "Đăng nhập".
 2. Hệ thống thực hiện truy vấn `SELECT * FROM users WHERE email = ?` để kiểm tra tài khoản.
 3. Hệ thống giải mã và đối chiếu mật khẩu nhập vào với chuỗi hash mã hóa trong CSDL.
 4. Đối chiếu trùng khớp thành công, hệ thống kiểm tra phân quyền:
Nếu `role = 'admin'`, điều hướng vào trung tâm quản trị `/admin/dashboard`; nếu `role = 'user'`, điều hướng về trang chủ `/dashboard`.
 - Luồng đăng nhập nhanh bằng tài khoản Google:
 1. Người dùng nhấn vào nút "Tiếp tục với Google" (được nhúng mã SVG đồ họa tĩnh).
 2. Ứng dụng thông qua thư viện Laravel Socialite gọi API Google OAuth 2.0 để mở cửa sổ xác thực an toàn.
 3. Người dùng chọn tài khoản Gmail, Google trả gói dữ liệu callback chứa `google_id`, email, và name về hệ thống.
 4. Hệ thống rà soát bảng dữ liệu: Nếu tìm thấy tài khoản có `google_id` hoặc email trùng khớp, hệ thống tự động cập nhật đồng bộ thông tin và đăng nhập thẳng vào trang Client.

- **Luồng sự kiện ngoại lệ (Exception Flow):**

- Trường hợp sai thông tin đăng nhập: Tại bước 3 của luồng truyền thống, nếu mật khẩu hoặc email không tồn tại, hệ thống chặn lại và thông báo: "Thông tin tài khoản hoặc mật khẩu không chính xác".
- Trường hợp người dùng mới đăng nhập Google lần đầu: Tại bước 4 của luồng Google, nếu thông tin email chưa từng có trong hệ thống, hệ thống tự động chạy lệnh tạo một bản ghi User mới sạch trên máy với mật khẩu ngẫu nhiên, cấp quyền role = 'user' và đăng nhập thẳng.

3.3. RQ03 – Đăng xuất tài khoản

- **Tên chức năng:** Đăng xuất tài khoản và kết thúc phiên làm việc.
- **Mô tả:** Hủy bỏ trạng thái đăng nhập, xóa các token bảo mật để bảo vệ an toàn cho tài khoản người dùng.
- **Tác nhân (Actor):** Tất cả người dùng đã đăng nhập hệ thống.
- **Tiền điều kiện (Pre-conditions):** Trạng thái phiên làm việc (Session) của tài khoản đang ở chế độ hoạt động (Active).
- **Hậu điều kiện (Post-conditions):** Cookie phiên làm việc trên trình duyệt bị xóa bỏ hoàn toàn, đưa hệ thống về trạng thái công khai vắng lai.
- **Luồng sự kiện chính (Basic Flow):**
 1. Người dùng click vào tên tài khoản hoặc Avatar trên thanh công cụ và chọn nút "Đăng xuất".
 2. Hệ thống tiếp nhận yêu cầu, kích hoạt cơ chế hủy bỏ (Flush/Forget) toàn bộ các biến trạng thái, dữ liệu định danh được lưu trữ tạm thời trong session.
 3. Hệ thống thực hiện thu hồi các khóa Token bảo mật chống giả mạo liên quan.
 4. Điều hướng Quản trị viên về trang đăng nhập Admin hoặc điều hướng Khách hàng về trang giới thiệu công khai của website.
- **Luồng sự kiện ngoại lệ (Exception Flow):** Không có.

3.4. RQ04 – Quản lý hồ sơ cá nhân (Profile)

- **Tên chức năng:** Xem, cập nhật thông tin cá nhân và thay đổi mật khẩu.
- **Mô tả:** Cho phép người dùng chỉnh sửa thông tin hiển thị hoặc tiến hành xóa bỏ tài khoản thành viên khỏi hệ thống.
- **Tác nhân (Actor):** Khách hàng thành viên / Quản trị viên.
- **Tiền điều kiện (Pre-conditions):** Tài khoản đã vượt qua lớp Middleware xác thực đăng nhập thành công.
- **Hậu điều kiện (Post-conditions):** Dữ liệu cập nhật mới được ghi đè an toàn vào bảng users.
- **Luồng sự kiện chính (Basic Flow):**
 1. Người dùng truy cập vào mục "Quản lý tài khoản" (route /profile). Giao diện gọi hàm ProfileController@edit tự động hiển thị thông tin Họ tên và Email hiện tại lên các ô nhập liệu.
 2. Người dùng thay đổi thông tin Họ tên, Số điện thoại hoặc điền vào Form thay đổi mật khẩu bảo mật.
 3. Người dùng nhấn chọn nút "Lưu thay đổi".
 4. Hệ thống kiểm tra dữ liệu đầu vào. Nếu dữ liệu hợp lệ, Laravel Eloquent thực thi lệnh Update cập nhật thông tin mới vào bảng dữ liệu của CSDL.
 5. Hệ thống làm mới giao diện hiển thị và đưa ra thông báo dạng Toast xanh báo hiệu cập nhật hồ sơ thành công.
- **Luồng sự kiện ngoại lệ (Exception Flow):**
 1. Trường hợp đổi mật khẩu sai mật khẩu hiện tại: Nếu người dùng nhập form đổi mật khẩu nhưng ô "Mật khẩu hiện tại" không trùng khớp với mật khẩu đang lưu trong CSDL, hệ thống chặn tiến trình và thông báo lỗi: "Mật khẩu hiện tại không chính xác".
 2. Trường hợp xóa tài khoản: Nếu người dùng chọn nút "Xóa tài khoản", hệ thống hiển thị hộp thoại yêu cầu nhập mật khẩu để xác nhận danh tính. Nếu nhập đúng, hệ thống kích hoạt lệnh DELETE xóa vĩnh viễn bản ghi khỏi bảng users và hủy session.

3.5. RQ05 – Xem và lọc xe theo phân khúc

- **Tên chức năng:** Xem danh mục phương tiện và lọc xe theo phân khúc du lịch.
- **Mô tả:** Hiển thị trực quan toàn bộ kho xe đang kinh doanh của nhà xe và hỗ trợ lọc xe nhanh theo số chỗ hoặc kiểu xe.
- **Tác nhân (Actor):** Khách hàng thành viên / Khách hàng vắng lai.
- **Tiền điều kiện (Pre-conditions):** Người dùng truy cập vào trang chủ giao diện Client (/dashboard).
- **Hậu điều kiện (Post-conditions):** Trình duyệt hiển thị danh sách xe tương ứng theo đúng nhu cầu tìm kiếm của người dùng.
- **Luồng sự kiện chính (Basic Flow):**
 1. Khách hàng truy cập vào giao diện trang chủ Client, hệ thống tự động chạy hàm HomeController@index.
 2. Hệ thống gửi lệnh `SELECT * FROM cars WHERE status != 'hidden'` xuống CSDL để lấy dữ liệu.
 3. Giao diện render danh sách xe theo cấu trúc dạng lưới (Grid Card) bao gồm các thông tin: Tên xe, Hãng xe, Loại hộp số, Nhiên liệu và Giá thuê theo ngày.
 4. Khách hàng click chọn vào các tab danh mục phân khúc hiển thị đầu trang (ví dụ: Phân khúc xe 4 chỗ - Sedan, Phân khúc xe 7 chỗ - SUV).
 5. Hệ thống tiếp nhận tham số ID danh mục, thực hiện truy vấn điều kiện kết nối khóa ngoại: `SELECT * FROM cars WHERE category_id = ?`.
 6. Lọc dữ liệu thành công, hệ thống cập nhật và hiển thị danh sách các xe thuộc phân khúc khách chọn lên màn hình.
- **Luồng sự kiện ngoại lệ (Exception Flow):**
 1. Trường hợp phân khúc chưa có xe: Nếu Admin mới tạo danh mục phân khúc nhưng chưa thực hiện Import file CSV hay thêm xe vào danh mục đó, hệ thống sẽ trả về màn hình trống kèm dòng chữ thông báo: "Hiện tại chưa có phương tiện nào thuộc phân khúc này. Vui lòng quay lại sau!".

3.6. RQ06 – Xem chi tiết thông số xe

- **Tên chức năng:** Hiển thị thông số kỹ thuật chi tiết của phương tiện.
- **Mô tả:** Cung cấp thông tin chi tiết về xe ô tô giúp khách hàng có đầy đủ cơ sở trước khi quyết định đặt xe thuê.
- **Tác nhân (Actor):** Khách hàng thành viên / Khách hàng vắng lai.
- **Tiền điều kiện (Pre-conditions):** Người dùng đang ở màn hình danh sách xe tổng quan (RQ05).
- **Hậu điều kiện (Post-conditions):** Hệ thống hiển thị toàn bộ thuộc tính chi tiết của phương tiện được chọn.
- **Luồng sự kiện chính (Basic Flow):**
 1. Khách hàng duyệt danh sách xe và click chuột vào hình ảnh hoặc tên của một chiếc xe cụ thể (ví dụ: Mazda 3 Luxury).
 2. Hệ thống thu nhận mã định danh ID của chiếc xe đó và điều hướng trình duyệt sang địa chỉ route chi tiết (/xe/{id}).
 3. Hàm xử lý HomeController@show tiếp nhận ID, thực hiện câu lệnh tìm kiếm bản ghi: `SELECT * FROM cars WHERE id = ? LIMIT 1`.
 4. Hệ thống bóc tách các thuộc tính từ CSDL đổ lên giao diện Blade bao gồm: Hình ảnh xe kích thước lớn, Tên xe, Thương hiệu, Kiểu hộp số (Số sàn/Số tự động), Loại nhiên liệu (Xăng/Dầu), Số chỗ ngồi và các mô tả tiện nghi đi kèm.
- **Luồng sự kiện ngoại lệ (Exception Flow):**
 1. Trường hợp xe không tồn tại hoặc đã bị xóa: Nếu người dùng cố tình nhập một mã ID ngẫu nhiên không có thật lên thanh địa chỉ (ví dụ: /xe/99999), hệ thống không tìm thấy bản ghi trong DB, lập tức kích hoạt cơ chế bắt lỗi và trả về giao diện báo lỗi tiêu chuẩn 404 Not Found - Phương tiện không tồn tại.

3.7. RQ07 – Đặt đơn thuê xe & Thanh toán (Mô phỏng)

- **Tên chức năng:** Tạo đơn đặt thuê xe tự lái trực tuyến và xử lý hóa đơn tạm tính.

- **Mô tả:** Tiếp nhận thông tin lịch trình thuê xe của khách hàng, tự động tính toán tổng chi phí và đưa đơn vào trạng thái chờ phê duyệt.
- **Tác nhân (Actor):** Khách hàng thành viên (Đã đăng nhập tài khoản).
- **Tiền điều kiện (Pre-conditions):** Chiếc xe được chọn phải đang ở trạng thái sẵn sàng phục vụ (status = 'available').
- **Hậu điều kiện (Post-conditions):** Một bản ghi hóa đơn mới được tạo trong bảng bookings với trạng thái ban đầu là pending.
- **Luồng sự kiện chính (Basic Flow):**
 1. Tại giao diện chi tiết xe, khách hàng nhấn chọn nút "Đặt xe ngay". Hệ thống điều hướng sang trang xác nhận thanh toán đặt chỗ tập trung (/thanh-toan/{id}).
 2. Khách hàng nhập các thông tin bắt buộc: Họ tên người lái, Số điện thoại liên hệ khẩn cấp, chọn Ngày nhận xe và Ngày trả xe.
 3. Hệ thống chạy mã JavaScript tự động kiểm tra khoảng cách giữa hai mốc ngày, nhân với đơn giá price_per_day của phương tiện và hiển thị tổng tiền hóa đơn tạm tính ra màn hình.
 4. Khách hàng kiểm tra thông tin số tiền và nhấn chọn nút "Xác nhận đặt đơn".
 5. Hệ thống gửi yêu cầu dữ liệu qua phương thức POST về route /dat-xe. Hàm BookingController@store tiếp nhận dữ liệu và thực hiện lệnh INSERT INTO bookings.
 6. Ghi dữ liệu thành công, hệ thống hiển thị màn hình thông báo Pop-up chúc mừng đặt đơn thành công và chuyển đơn sang hàng đợi phê duyệt của Admin.
- **Luồng sự kiện ngoại lệ (Exception Flow):**
 1. Trường hợp chọn sai niên độ ngày nhận/trả: Nếu khách hàng chọn ngày trả xe trước ngày nhận xe hoặc chọn các ngày đã qua trong quá khứ, hệ thống JavaScript và bộ lọc dữ liệu Backend sẽ chặn lại, không cho bấm nút xác

nhận và hiển thị thông báo: "Ngày thuê xe không hợp lệ. Vui lòng kiểm tra lại!".

3.8. RQ08 – Xem lịch sử đặt xe cá nhân

- **Tên chức năng:** Tra cứu lịch sử đặt đơn xe của cá nhân.
- **Mô tả:** Giúp khách hàng theo dõi tiến độ phê duyệt hoặc xem lại toàn bộ các giao dịch đặt thuê xe mà mình đã thực hiện từ trước tới nay.
- **Tác nhân (Actor):** Khách hàng thành viên.
- **Tiền điều kiện (Pre-conditions):** Tài khoản khách hàng đang ở trạng thái đăng nhập hoạt động.
- **Hậu điều kiện (Post-conditions):** Danh sách toàn bộ đơn đặt xe của riêng tài khoản đó được trích xuất hiển thị lên màn hình.
- **Luồng sự kiện chính (Basic Flow):**
 1. Khách hàng nhấn chọn vào mục "Lịch sử đặt đơn" trên thanh menu tài khoản cá nhân.
 2. Hệ thống tiếp nhận yêu cầu, lấy ID của người dùng hiện tại từ Session làm bộ lọc điều kiện.
 3. Hệ thống gọi hàm `BookingController@history` thực hiện câu lệnh truy vấn có điều kiện: `SELECT * FROM bookings WHERE user_id = ? ORDER BY created_at DESC`.
 4. CSDL trả về danh sách các bản ghi, hệ thống tiến hành render dữ liệu lên bảng lịch sử theo cấu trúc: Mã đơn hàng, Tên xe, Ngày thuê, Tổng chi phí và Trạng thái đơn (được tô màu nhận diện riêng biệt: Màu vàng cho Chờ duyệt, Màu xanh cho Đã duyệt, Màu đỏ cho Đã hủy).
- **Luồng sự kiện ngoại lệ (Exception Flow):** Không có.

3.9. RQ09 – Gửi tin nhắn liên hệ hỗ trợ

- **Tên chức năng:** Gửi form thông tin liên hệ và để lại số điện thoại hỗ trợ tư vấn.
- **Mô tả:** Phân hệ dành cho khách hàng cần giải đáp thắc mắc, đàm phán thủ tục nhận xe hoặc yêu cầu nhân viên gọi điện lại để làm việc trực tiếp.

- **Tác nhân (Actor):** Khách hàng thành viên / Khách hàng vắng lai.
- **Tiền điều kiện (Pre-conditions):** Người dùng truy cập vào trang Liên hệ của ứng dụng (/lien-he).
- **Hậu điều kiện (Post-conditions):** Thông tin lời nhắn và số điện thoại của khách được lưu thành công vào bảng dữ liệu tin nhắn liên hệ trong CSDL trên Railway.
- **Luồng sự kiện chính (Basic Flow):**
 1. Người dùng truy cập giao diện trang Liên hệ, hệ thống gọi hàm `ContactController@index` hiển thị Form liên hệ.
 2. Người dùng tiến hành điền đầy đủ các thông tin vào form bao gồm: Họ và tên, Địa chỉ Email, Số điện thoại di động liên lạc và nội dung chi tiết câu hỏi cần hỗ trợ tư vấn.
 3. Người dùng kiểm tra thông tin và nhấn chọn nút "Gửi thông tin liên hệ".
 4. Hệ thống đóng gói dữ liệu và gửi qua phương thức POST lên route `/contact/submit`.
 5. Hàm `ContactController@store` kích hoạt kiểm tra Validation đầu vào, đảm bảo các trường bắt buộc không được để trống.
 6. Dữ liệu hợp lệ, hệ thống thực thi lệnh lưu bản ghi mới vào CSDL và hiển thị thông báo thông minh: "Thông tin của bạn đã được gửi thành công. Nhân viên CARENTO sẽ chủ động liên hệ lại làm việc với bạn qua điện thoại trong ít phút!".
- **Luồng sự kiện ngoại lệ (Exception Flow):**
 1. Trường hợp thiếu thông tin liên lạc cốt lõi: Tại bước 5, nếu người dùng bỏ trống trường Số điện thoại hoặc trường nội dung lời nhắn, hệ thống lập tức chặn lại, không ghi vào CSDL và hiển thị cảnh báo đỏ cạnh ô nhập: "Vui lòng nhập số điện thoại để nhân viên gọi lại hỗ trợ" hoặc "Nội dung liên hệ không được để trống".

3.10. RQ10 – Báo cáo thống kê tổng quan (Admin Dashboard)

- **Tên chức năng:** Thống kê chỉ số kinh doanh và hiệu suất vận hành tại trung tâm điều hành Admin.

- **Mô tả:** Tự động tổng hợp số liệu tài chính tổng doanh thu, tổng đơn hàng từ cơ sở dữ liệu để phục vụ cho công tác quản trị của chủ nhà xe.
- **Tác nhân (Actor):** Quản trị viên (Admin).
- **Tiền điều kiện (Pre-conditions):** Tài khoản đã đăng nhập thành công và được xác thực quyền quản trị tối cao bởi AdminMiddleware.
- **Hậu điều kiện (Post-conditions):** Giao diện Dashboard hiển thị số liệu thống kê tài chính chính xác theo thời gian thực.
- **Luồng sự kiện chính (Basic Flow):**
 1. Admin truy cập vào đường dẫn quản trị hệ thống tại địa chỉ route /admin/dashboard.
 2. Hệ thống tiếp nhận yêu cầu, kích hoạt lớp xử lý dữ liệu tổng hợp DashboardController@index.
 3. Hệ thống thực hiện câu lệnh tính tổng doanh thu từ các đơn hàng đã hoàn thành: `SELECT SUM(total_price) FROM bookings WHERE status = 'approved'`.
 4. Hệ thống tiếp tục thực hiện các câu lệnh đếm tổng số lượng đơn hàng, số lượng khách viên và số lượng tin nhắn liên hệ chưa xử lý.
 5. Hệ thống tổng hợp tất cả các chỉ số kết quả, đổ dữ liệu vào các thẻ Grid đồ họa trực quan và render màn hình báo cáo hoàn chỉnh cho Admin theo dõi.
- **Luồng sự kiện ngoại lệ (Exception Flow):** Không có.

3.11. RQ11 – Quản lý kho xe - CRUD Cars (Admin)

- **Tên chức năng:** Quản lý kho phương tiện và cập nhật trạng thái vận hành xe nhanh.
- **Mô tả:** Cho phép Admin thực hiện đầy đủ các chức năng thêm, sửa, xóa thông số phương tiện, hoặc thực hiện đổi trạng thái bảo dưỡng của xe ngay tại màn hình danh sách.
- **Tác nhân (Actor):** Quản trị viên (Admin).
- **Tiền điều kiện (Pre-conditions):** Đăng nhập quyền Admin, đang ở phân hệ quản lý tài sản xe (/admin/cars).

- **Hậu điều kiện (Post-conditions):** Dữ liệu kho xe ô tô trong bảng cars được cập nhật biến động chính xác.
- **Luồng sự kiện chính (Basic Flow):**
 - Luồng Thêm/Sửa phương tiện truyền thống:
 1. Admin mở form Thêm hoặc Sửa xe, nhập các thông số kỹ thuật (Tên xe, Hãng, Hộp số, Nhiên liệu, Đơn giá thuê) và chọn phân khúc danh mục xe.
 2. Hệ thống kiểm tra ràng buộc khóa ngoại: Đảm bảo category_id của phân khúc xe tồn tại thực tế trong bảng danh mục categories. Nếu thông qua, thực hiện câu lệnh ghi đè hoặc khởi tạo bản ghi vào bảng cars.
 - Luồng cập nhật nhanh trạng thái vận hành trên Grid:
 1. Tại bảng lưới danh sách xe tổng quát, Admin click chuột vào nút gạt chuyển đổi trạng thái (Toggle/Button status) của một chiếc xe cụ thể.
 2. Hệ thống bắt nhận ID xe và kích hoạt phương thức truyền tải PATCH gửi yêu cầu trực tiếp về route /admin/cars/{id}/status.
 3. Hàm CarController@updateStatus tiếp nhận yêu cầu, lập tức chạy câu lệnh cập nhật đảo ngược giá trị trường trạng thái hiện tại (từ available sang maintenance để đưa xe đi bảo dưỡng định kỳ hoặc ngược lại).
 4. Hệ thống thực hiện lưu thay đổi và cập nhật màu sắc cờ hiệu trạng thái trên bảng lưới lập tức mà không cần tải lại trang.
- **Luồng sự kiện ngoại lệ (Exception Flow):**
 - Trường hợp vi phạm khóa ngoại: Khi thêm xe, nếu Admin vô tình chọn một danh mục phân khúc chưa được khởi tạo cấu trúc id trong bảng categories, hệ thống database sẽ chặn lại và văng lỗi ràng buộc liên kết. Hệ thống Laravel lập tức bắt lấy ngoại lệ này bằng khối try-catch, chặn tiến trình ghi dữ liệu rác và xuất thông báo cảnh báo Admin: "Danh mục xe không hợp lệ. Vui lòng kiểm tra lại cấu trúc danh mục!".

3.12. RQ12 – Quản lý danh sách thành viên khách hàng

- **Tên chức năng:** Xem và rà soát danh sách thành viên hệ thống.
- **Mô tả:** Cung cấp cho Admin công cụ quản lý toàn bộ tệp khách hàng đã đăng ký tài khoản thành viên, hỗ trợ theo dõi mức độ tương tác đặt đơn của từng người.
- **Tác nhân (Actor):** Quản trị viên (Admin).
- **Tiền điều kiện (Pre-conditions):** Đăng nhập quyền Admin, truy cập vào đường dẫn route /admin/customers.
- **Hậu điều kiện (Post-conditions):** Hệ thống hiển thị bảng danh sách thông tin hồ sơ của toàn bộ người dùng hệ thống.
- **Luồng sự kiện chính (Basic Flow):**
 1. Admin nhấn chọn phân hệ "Quản lý khách hàng" trên thanh Menu điều hướng quản trị.
 2. Hệ thống kích hoạt hàm xử lý dữ liệu CustomerController@index thực hiện câu lệnh truy vấn có bộ lọc phân quyền: `SELECT * FROM users WHERE role = 'user' ORDER BY created_at DESC`.
 3. Đồng thời, hệ thống chạy câu lệnh kết hợp đếm số lượng đơn đặt hàng (bookings_count) tương ứng kết nối theo ID của từng khách hàng từ bảng bookings.
 4. Hệ thống tổng hợp dữ liệu kết quả, áp dụng định dạng giao diện chuyên biệt từ tệp phong cách thiết kế riêng customers.css và render bảng danh sách hiển thị mượt mà lên màn hình máy Mac/PC của Admin.
- **Luồng sự kiện ngoại lệ (Exception Flow):** Không có.

3.13. RQ13 – Điều phối duyệt đơn đặt xe (Admin Bookings)

- **Tên chức năng:** Phê duyệt hoặc hủy đơn đặt xe thuê trực tuyến của khách hàng.
- **Mô tả:** Quy trình tiếp nhận đơn hàng, rà soát lịch trình và thực hiện điều phối phê duyệt hoặc từ chối đơn hàng của khách hàng thành viên.
- **Tác nhân (Actor):** Quản trị viên (Admin).
- **Tiền điều kiện (Pre-conditions):** Hệ thống có đơn đặt xe mới ở trạng thái chờ xử lý (status = 'pending').

- **Hậu điều kiện (Post-conditions):** Trạng thái đơn đặt xe trong bảng bookings thay đổi, trạng thái xe trong bảng cars được đồng bộ hóa tương ứng.
- **Luồng sự kiện chính (Basic Flow):**
 1. Admin truy cập vào trang Quản lý đơn đặt xe tại địa chỉ route /admin/bookings.
 2. Hệ thống hiển thị bảng danh sách các đơn hàng. Admin chọn xem một đơn hàng cụ thể, rà soát thông tin: Tên khách, số điện thoại, dòng xe, thời gian thuê và tổng tiền hóa đơn.
 3. Admin tiến hành đưa ra quyết định điều phối xử lý đơn hàng thông qua phương thức PATCH gửi về route /admin/bookings/{id}/status:
 - Nếu chọn nút Phê duyệt (Approve): Hệ thống cập nhật trường trạng thái của đơn hàng thành approved. Đồng thời, hệ thống tự động chạy lệnh cập nhật đồng bộ, chuyển trạng thái của chiếc xe tương ứng trong bảng cars sang mã trạng thái rented để khóa phương tiện, ngăn chặn tuyệt đối tình trạng khách hàng khác đặt trùng lịch xe này.
 - Nếu chọn nút Từ chối/Hủy đơn (Cancel): Hệ thống cập nhật trường trạng thái đơn thành cancelled và giải phóng phương tiện về trạng thái sẵn sàng.
 4. Hệ thống tự động chạy hàm cập nhật làm mới lại các chỉ số tổng doanh thu thực tế trên màn hình Dashboard chính ngay sau khi thao tác thành công.
- **Luồng sự kiện ngoại lệ (Exception Flow):** Không có.

3.14. RQ14 – Quản lý tin nhắn liên hệ (Admin Messages)

- **Tên chức năng:** Tiếp nhận, rà soát thông tin tin nhắn liên hệ gọi lại tư vấn của khách hàng.
- **Mô tả:** Phân hệ hiển thị toàn bộ danh sách số điện thoại và lời nhắn của khách hàng gửi từ module liên hệ (RQ09) để nhân viên trực tổng đài nhắc máy điện thoại gọi lại làm việc và chốt hợp đồng trực tiếp.
- **Tác nhân (Actor):** Quản trị viên (Admin) / Nhân viên trực tổng đài nhà xe.

- **Tiền điều kiện (Pre-conditions):** Khách hàng đã thực hiện gửi form thông tin liên hệ thành công ở giao diện ngoài Client.
- **Hậu điều kiện (Post-conditions):** Các yêu cầu tư vấn được rà soát và xử lý thông suốt thông qua kênh liên lạc trực tiếp.
- **Luồng sự kiện chính (Basic Flow):**
 1. Admin hoặc nhân viên tổng đài nhà xe truy cập vào mục "Quản lý tin nhắn" tại đường dẫn route /admin/messages.
 2. Hệ thống kích hoạt hàm xử lý MessageController@index, thực hiện câu lệnh truy vấn dữ liệu từ bảng tin nhắn liên hệ trong CSDL và sắp xếp hiển thị theo cấu trúc thời gian gửi mới nhất lên trên đầu.
 3. Hệ thống nạp cấu trúc định dạng giao diện chuyên biệt từ tệp messages.css để render bảng hiển thị rõ ràng các cột thông tin: Họ và tên khách hàng, Email, Số điện thoại liên lạc khẩn cấp và nội dung chi tiết lời nhắn yêu cầu hỗ trợ tư vấn.
 4. **Nghiệp vụ xử lý offline thực tế của doanh nghiệp:** Nhân viên trực tổng đài nhìn vào bảng dữ liệu thông tin hiển thị trên màn hình, **tiến hành sử dụng điện thoại trực tiếp bấm số gọi điện lại cho khách hàng** theo số điện thoại khách để lại trên form. Nhân viên tiến hành tư vấn thủ tục, đàm phán giá cả, hẹn lịch nhận xe và hoàn tất chốt hợp đồng thuê xe offline trực tiếp với đối tác.
- **Luồng sự kiện ngoại lệ (Exception Flow):** Không có.

4. CÁC YÊU CẦU PHI CHỨC NĂNG

4.1. Yêu cầu về hiệu năng thực thi (Performance Requirements)

Yêu cầu hiệu năng định cấu hình các chỉ số kỹ thuật về tốc độ xử lý, khả năng chịu tải và tối ưu hóa tài nguyên phần mềm ở cả hai đầu hệ thống:

- **Tốc độ phản hồi của hệ thống (Response Time):** Mọi thao tác gửi dữ liệu từ Form biểu mẫu của khách hàng (Form gửi thông tin liên lạc tại ContactController@store hay Form xác nhận đặt xe tại BookingController@store) bắt buộc phải được xử lý và trả về kết quả thành

công trong thời gian giới hạn dưới 1.5 giây. Đối với các trang hiển thị danh sách tổng quan (/dashboard) hoặc trang chi tiết xe (/xe/{id}), thời gian phản hồi lý tưởng từ máy chủ đám mây Railway về trình duyệt máy khách phải dưới 1 giây nhằm tối ưu hóa trải nghiệm giữ chân khách hàng.

- Tối ưu hóa các câu lệnh truy vấn dữ liệu (Database Query Optimization): Khi Quản trị viên truy cập vào trung tâm điều hành /admin/dashboard, hệ thống phải thực hiện đồng thời các hàm tính toán phức tạp như SUM(total_price) trên bảng bookings và các hàm COUNT kết nối đa bảng (Join Queries) để đếm số lượng đơn đặt xe, lượng thành viên người dùng và số tin nhắn liên hệ. Các thuật toán này bắt buộc phải được tối ưu hóa thông qua việc đánh chỉ mục (Indexing) các trường khóa chính, khóa ngoại, đảm bảo thời gian thực thi câu lệnh SQL dưới Database diễn ra dưới 500 mili giây.
- Tối ưu hóa tài nguyên Front-end (Asset Minification): Toàn bộ các tệp tin cấu hình giao diện Tailwind CSS và logic JavaScript điều khiển biểu mẫu bắt buộc phải được đóng gói nén gọn, xóa bỏ hoàn toàn khoảng trắng và các dòng ghi chú dư thừa thông qua quy trình biên dịch của trình chạy Vite Compiler. Việc băm nhỏ và nén asset này giúp giảm dung lượng băng thông tải về máy khách xuống mức tối thiểu, tăng tốc độ render giao diện trên các thiết bị di động có kết nối mạng di động (3G/4G/5G) yếu.

4.2. Yêu cầu về độ tin cậy và xử lý ngoại lệ (Reliability & Exception Handling)

Yêu cầu này đảm bảo hệ thống vận hành ổn định liên tục, có khả năng tự phục hồi và bảo vệ dữ liệu khi đối mặt với các lỗi phát sinh trong môi trường mạng:

- Cơ chế bắt lỗi ngoại lệ chặt chẽ (Robust Exception Handling): Mã nguồn tại tất cả các file xử lý logic Controller (ContactController, BookingController, CarController, CustomerController) bắt buộc phải được bọc trong các khối lệnh điều khiển try-catch. Khi xảy ra các sự cố ngoài ý muốn (ví dụ: Mất kết nối Database tạm thời, lỗi ghi dữ liệu trùng lặp), hệ thống Laravel phải bắt lấy ngoại lệ này, tự động ghi nhật ký lỗi (System Logs)

phục vụ bảo trì, tuyệt đối không được văng các dòng mã nguồn PHP thô hoặc lỗi cấu trúc SQL 500 Internal Server Error ra màn hình trình duyệt của khách hàng.

- Xử lý giao diện báo lỗi thông minh (Friendly Error Pages): Khi xảy ra lỗi ngoại lệ, hệ thống phải điều hướng người dùng sang các trang báo lỗi giao diện được thiết kế thân thiện (như trang báo lỗi 404 khi truy cập sai ID xe, hoặc trang báo lỗi hệ thống tạm thời), kèm theo nút hướng dẫn quay trở lại trang chủ để tránh làm đứt gãy luồng trải nghiệm của người dùng.
- Đảm bảo tính toàn vẹn dữ liệu (Data Integrity Constraints): Khi Quản trị viên thực hiện thao tác Phê duyệt đơn đặt xe (RQ13), quy trình chuyển đổi trạng thái đơn hàng sang approved trong bảng bookings và quy trình cập nhật chuyển trạng thái của chiếc xe tương ứng sang rented trong bảng cars phải được thực hiện đồng bộ theo cơ chế giao dịch nguyên tử (Database Transactions). Nếu một trong hai tiến trình bị lỗi, toàn bộ trạng thái phải được quay lui (Rollback) về ban đầu để tránh hiện tượng vỡ dữ liệu lịch trình xe.
- Chỉ số hoạt động liên tục (System Uptime): Hệ thống ứng dụng container ảo hóa của server đám mây Railway, đảm bảo duy trì tần suất hoạt động liên tục ổn định với chỉ số Uptime đạt trên 99.9%, có khả năng tự động khởi động lại dịch vụ (Auto-restart) lập tức khi xảy ra hiện tượng tràn bộ nhớ.

4.3. Yêu cầu về bảo mật an toàn thông tin (Security Requirements)

Yêu cầu bảo mật thiết lập các rào cản kỹ thuật để chống lại các cuộc tấn công phá hoại và lộ lọt dữ liệu riêng tư của khách hàng ra bên ngoài:

- Cô lập thông tin nhạy cảm (Environment Isolation): Toàn bộ các mã khóa an ninh cốt lõi của dự án (như chuỗi ký tự mật mã hệ thống APP_KEY, cặp mã xác thực GOOGLE_CLIENT_ID và GOOGLE_CLIENT_SECRET dùng cho Google OAuth 2.0) tuyệt đối không được viết trực tiếp (Hardcode) vào các file mã nguồn. Các chuỗi bảo mật này bắt buộc phải được lưu trữ tập trung tại phân hệ biến môi trường Variables trên giao diện quản trị của mây Railway và được mã hóa an toàn, cách ly hoàn toàn khỏi kho chứa mã nguồn công khai trên GitHub Repository.

- Phân quyền truy cập nghiêm ngặt bằng Middleware: Hệ thống sử dụng lớp lọc AdminMiddleware để thiết lập rào cản kiểm soát an ninh đối với phân hệ backend /admin/*. Toàn bộ thông tin hồ sơ khách hàng, số điện thoại, danh sách đơn đặt và nội dung tin nhắn liên hệ của khách hàng lưu trong CSDL thuộc diện bảo mật cao. Hệ thống sẽ chặn đứng và trả về lỗi từ chối truy cập 403 Forbidden đối với bất kỳ tài khoản nào không phải là Admin cố tình truy cập vào các route nhạy cảm như xem danh sách khách hàng (/admin/customers) hay quản lý tin nhắn liên hệ (/admin/messages).
- Mã hóa mật khẩu một chiều: Mật khẩu của người dùng khi đăng ký tài khoản bắt buộc phải được mã hóa bằng thuật toán băm một chiều Bcrypt trước khi lưu trữ vào bảng users, đảm bảo ngay cả khi cơ sở dữ liệu bị hacker tấn công trực xuất dữ liệu thô, mật khẩu gốc của người dùng vẫn được bảo vệ an toàn tuyệt đối.
- Chống tấn công giả mạo yêu cầu chéo trường (CSRF Protection): Hệ thống áp dụng cơ chế tự động đính kèm mã định danh Token chống giả mạo (@csrf) trên tất cả các thẻ form biểu mẫu sử dụng các phương thức ghi dữ liệu POST và PATCH, bảo vệ website CARENTO khỏi các cuộc tấn công chiếm đoạt quyền gửi tin nhắn rác hoặc tạo đơn hàng giả mạo từ bên thứ ba.

4.4. Yêu cầu về chất lượng và tính tiện dụng (Usability & Quality Requirements)

Yêu cầu này định hình tính thẩm mỹ, độ hoàn thiện kỹ thuật và khả năng tương tác UI/UX tối ưu của ứng dụng đối với người dùng cuối:

- Thiết kế Responsive tương thích đa nền tảng: Giao diện Form liên hệ hỗ trợ gửi tin nhắn và Form thanh toán đặt xe bắt buộc phải được thiết kế co giãn chuẩn mực theo ma trận pixel của thiết bị di động thông qua các class tiện ích của Tailwind CSS. Các ô nhập liệu họ tên, email, nội dung tin nhắn và đặc biệt là ô nhập Số điện thoại phải có kích thước lớn, khoảng cách đệm (Padding) rộng, giúp khách hàng dễ dàng thao tác gõ nhập thông tin bằng một tay khi đang di chuyển trên đường một cách thuận tiện.
- Đồng bộ hóa phong chữ tiếng Việt hệ thống: Hệ thống loại bỏ hoàn toàn các phong chữ mặc định bị lỗi dấu tiếng Việt bằng cách nhúng trực tiếp thư viện

Google Fonts và ép toàn bộ các thẻ thành phần trên trang web (body, input, button, textarea, span...) dùng chung một phông chữ hiện đại duy nhất là Inter đi kèm thuộc tính !important. Giải pháp này giúp chữ viết nét căng, đồng đều, không bị hiện tượng nét đậm nét nhạt khi hiển thị các ký tự tiếng Việt có dấu phức tạp như ố, ệ, ử... trên mọi hệ điều hành từ macOS, Windows cho đến iOS và Android.

- Tính dễ bảo trì và mở rộng cấu trúc (Maintainability): Mã nguồn dự án được tổ chức khoa học theo chuẩn mô hình MVC tách biệt rõ ràng giữa các lớp. Sự liên kết lỏng lẻo giữa các module giúp mã nguồn sạch, dễ đọc. Cấu trúc này đảm bảo trong tương lai, khi hệ thống cần nâng cấp các tính năng nâng cao (ví dụ: Tích hợp thêm cổng thanh toán dòng tiền thật VNPAY, MoMo vào route /thanh-toan/{id}, hoặc tích hợp bản đồ số Google Maps API vào trang chi tiết xe), các lập trình viên thế hệ sau có thể dễ dàng cài đặt thư viện mới mà không làm ảnh hưởng đến cấu trúc khung xương vận hành cốt lõi hiện có của CARENTO.

CHƯƠNG 2: THIẾT KẾ VÀ CÀI ĐẶT GIẢI PHÁP

1. KIẾN TRÚC HỆ THỐNG

1.1. Thiết kế kiến trúc tổng quan

Hệ thống quản lý và đặt thuê xe tự lái trực tuyến CARENTO được xây dựng và phát triển dựa trên mô hình kiến trúc MVC (Model - View - Controller) tiêu chuẩn của Framework Laravel. Đây là một kiến trúc phân lớp mạnh mẽ, giúp tách biệt hoàn toàn giữa dữ liệu hệ thống, logic xử lý nghiệp vụ và giao diện hiển thị nhằm tăng cường tính bảo mật, dễ bảo trì và mở rộng mã nguồn.

Tầng Định tuyến (Routing Layer): Tiếp nhận toàn bộ các yêu cầu HTTP Requests (GET, POST, PATCH) từ trình duyệt của khách hàng gửi lên máy chủ Railway. Tầng này đóng vai trò điều hướng, kiểm tra tính hợp lệ của đường dẫn và áp dụng bộ lọc an ninh AdminMiddleware hoặc auth trước khi chuyển tiếp yêu cầu đến các lớp xử lý sâu hơn.

Tầng Điều khiển (Controller Layer): Nơi tập trung toàn bộ logic nghiệp vụ (Business Logic) của hệ thống CARENTO. Lớp này tiếp nhận tham số từ Request, ra lệnh cho lớp Model truy vấn dữ liệu, thực hiện các thuật toán tính toán (ví dụ: tính toán tổng chi phí thuê xe dựa trên số ngày tại BookingController) và chuẩn bị tài nguyên trả về cho tầng hiển thị.

Tầng Mô hình dữ liệu (Model Layer): Tương tác trực tiếp với hệ quản trị cơ sở dữ liệu MySQL thông qua công cụ ánh xạ Eloquent ORM. Lớp này quản lý cấu trúc thực thể, định nghĩa các mối quan hệ quan hệ giữa các bảng (Relationships) và thực hiện các quy tắc bảo mật dữ liệu đầu vào thông qua thuộc tính \$fillable.

Tầng Hiển thị (View Layer): Sử dụng hệ thống công cụ Blade Template Engine phối hợp cùng Tailwind CSS. Tầng này nhận dữ liệu tinh sạch từ Controller để render thành cấu trúc mã HTML5, CSS3 hoàn chỉnh, hiển thị trực quan lên màn hình thiết bị của người dùng cuối.

1.2. Mô tả sự phân rã hệ thống (Component Decomposition)

Hệ thống được bóc tách và phân rã thành ba phân hệ thành phần cốt lõi hoạt động đồng bộ với nhau:

1.2.1. Phân hệ Giao diện máy khách (Client-side Frontend)

Chịu trách nhiệm tương tác và thu thập hành vi của người dùng:

- Tài nguyên thiết kế công nghệ: Ứng dụng các class tiện ích của Tailwind CSS để tạo giao diện phẳng Responsive hiện đại. Hệ thống font chữ được ép đồng bộ qua tệp app.css sang font Inter để loại bỏ hoàn toàn lỗi hiển thị dấu tiếng Việt.
- Hệ thống quản lý Assets: Sử dụng Vite Compiler để quản lý tiến trình nạp tệp đồ họa tĩnh. Vite ánh xạ các trang chức năng đặc thù của Admin với các file CSS riêng biệt (customers.css cho quản lý khách hàng, messages.css cho quản lý tin nhắn liên hệ), giúp tối ưu dung lượng băng thông mạng.

1.2.2. Phân hệ Xử lý trung tâm (Server-side Backend)

Lỗi vận hành chạy trên máy chủ đám mây ảo hóa của Railway:

- Module Xác thực (Authentication Module): Ứng dụng gói thư viện Laravel Breeze bảo mật mã hóa phiên đăng nhập truyền thống, kết hợp thư viện Socialite để xử lý giao thức mã hóa dòng Token từ API Google Cloud Console, cho phép đăng nhập nhanh qua Google OAuth 2.0.
- Module Điều phối nghiệp vụ (Core Controllers): Phân tách thành hai vùng nhiệm vụ rõ rệt:
 - Vùng Client: HomeController (hiển thị, lọc danh mục xe), BookingController (xử lý luồng đặt xe và hóa đơn tạm tính), ContactController (tiếp nhận form liên hệ, để lại số điện thoại hỗ trợ).
 - Vùng Admin: DashboardController (tính toán doanh thu, lượng đơn hàng), CarController (CRUD kho xe và thay đổi trạng thái xe nhanh qua PATCH), CustomerController (quản lý thông tin hồ sơ thành viên), MessageController (tiếp nhận thông tin gọi lại hỗ trợ).

1.2.3. Phân hệ Lưu trữ (Database Storage Layer)

Hệ quản trị cơ sở dữ liệu quan hệ MySQL v8.0 được liên kết với máy chủ web thông qua chuỗi kết nối an toàn được cấu hình trong mục Variables của Railway. Hệ thống quản lý dữ liệu tập trung thông qua 5 thực thể quan hệ cốt lõi, ràng buộc chặt chẽ bằng các khóa ngoại (Foreign Keys) bảo vệ an toàn liên kết hệ thống.

2. THIẾT KẾ DỮ LIỆU

2.1. Mô hình dữ liệu quan hệ (Database Schema)

Cơ sở dữ liệu của hệ thống CARENTO được chuẩn hóa cấu trúc để tối ưu hóa tốc độ truy vấn và loại bỏ hiện tượng dư thừa dữ liệu. Mỗi quan hệ giữa các thực thể trong hệ thống được xác lập cụ thể như sau:

- Một Danh mục phân khúc (categories) sẽ chứa một hoặc nhiều phương tiện ô tô (cars) thuộc phân khúc đó (Mỗi quan hệ 1 - Nhiều). Thực thể cars giữ khóa ngoại category_id tham chiếu đến bảng categories.
- Một Người dùng (users) có thể thực hiện khởi tạo một hoặc nhiều đơn đặt thuê xe (bookings) trong suốt vòng đời tài khoản (Mỗi quan hệ 1 - Nhiều). Thực thể bookings giữ khóa ngoại user_id tham chiếu đến bảng users.
- Một Phương tiện xe (cars) có thể được đặt trong nhiều đơn đặt xe (bookings) theo các mốc lịch trình thời gian khác nhau (Mỗi quan hệ 1 - Nhiều). Thực thể bookings giữ khóa ngoại car_id tham chiếu đến bảng cars.
- Bảng Tin nhắn liên hệ hỗ trợ (contacts) hoạt động độc lập, lưu trữ dữ liệu các yêu cầu cần tư vấn khẩn cấp từ khách hàng để Admin tiếp nhận xử lý.

2.2. Từ điển dữ liệu chi tiết (Data Dictionary)

Dưới đây là cấu trúc đặc tả chi tiết của từng bảng dữ liệu thực tế được cài đặt trong hệ quản trị cơ sở dữ liệu MySQL của ứng dụng CARENTO trên môi trường Production:

2.2.1. Bảng users (Lưu trữ thông tin tài khoản người dùng hệ thống)

Bảng này dùng để lưu trữ toàn bộ thông tin đăng ký định danh của cả Khách hàng và Quản trị viên, hỗ trợ xác thực kép thông qua mật khẩu bấm hoặc mã nhận diện từ Google API.

STT	Tên trường (Field)	Kiểu dữ liệu (Type)	Ràng buộc (Constraints)
1	id	bigint unsigned	Primary Key, Auto Increment
2	name	varchar(255)	Not Null
3	email	varchar(255)	Not Null, Unique
4	email_verified_at	timestamp	Nullable
5	password	varchar(255)	Nullable
6	google_id	varchar(255)	Nullable, Unique
7	role	varchar(50)	Not Null, Default: 'user'
8	remember_token	varchar(100)	Nullable
9	created_at	timestamp	Nullable
10	updated_at	timestamp	Nullable

Bảng 2.1: Bảng dữ liệu User

2.2.2. Bảng categories (Quản lý danh mục phân khúc phương tiện)

Bảng lưu trữ các danh mục phân loại xe ô tô du lịch để phục vụ cho bộ lọc tìm kiếm tại giao diện Client.

STT	Tên trường (Field)	Kiểu dữ liệu (Type)	Ràng buộc (Constraints)	Mô tả chi tiết ý nghĩa trường
1	id	bigint unsigned	Primary Key, Auto Increment	Mã định danh duy nhất của danh mục xe.
2	name	varchar(255)	Not Null	Tên danh mục phân khúc (Sedan, SUV, Hatchback...).
3	created_at	timestamp	Nullable	Thời điểm danh mục được tạo mới.
4	updated_at	timestamp	Nullable	Thời điểm danh mục được cập nhật.

Bảng 2.2: Bảng dữ liệu Categories

2.2.3. Bảng cars (Quản lý thông tin chi tiết phương tiện ô tô)

Bảng chứa toàn bộ các thuộc tính kỹ thuật, đơn giá thuê và trạng thái vận hành của từng chiếc xe trong kho tài sản.

STT	Tên trường (Field)	Kiểu dữ liệu (Type)	Ràng buộc (Constraints)	Mô tả chi tiết ý nghĩa trường
1	id	bigint unsigned	Primary Key, Auto Increment	Mã định danh duy nhất của phương tiện xe.
2	category_id	bigint unsigned	Foreign Key	Khóa ngoại kết nối tham chiếu đến bảng categories.
3	name	varchar(255)	Not Null	Tên thương mại của phương tiện (Ví dụ: Mazda 3).
4	brand	varchar(255)	Not Null	Hãng sản xuất của xe (Toyota, Honda, Mazda...).
5	price_per_day	decimal(15,2)	Not Null	Đơn giá niêm yết tính theo 01 ngày thuê (VNĐ).
6	fuel_type	varchar(50)	Not Null	Loại nhiên liệu xe vận hành (Xăng / Dầu).
7	transmission	varchar(50)	Not Null	Kiểu hộp số điều khiển (Số sàn / Số tự động).
8	seats	int	Not Null	Số lượng chỗ ngồi thiết kế của xe (4

				chỗ, 7 chỗ).
9	image	varchar(255)	Nullable	Đường dẫn lưu trữ tệp tin ảnh xe trong thư mục asset.
10	status	varchar(50)	Default: 'available'	Trạng thái vận hành xe ('available', 'maintenance').
11	created_at	timestamp	Nullable	Thời điểm phương tiện được khai báo vào hệ thống.
12	updated_at	timestamp	Nullable	Thời điểm cập nhật lại thông số kỹ thuật xe.

Bảng 2.3: Bảng dữ liệu Cars

2.2.4. Bảng bookings (Quản lý thông tin chi tiết đơn đặt thuê xe trực tuyến)

Bảng cốt lõi lưu trữ dữ liệu giao dịch hóa đơn, lịch trình nhận/trả xe và các thông tin liên lạc khẩn cấp của khách đặt.

STT	Tên trường (Field)	Kiểu dữ liệu (Type)	Ràng buộc (Constraints)	Mô tả chi tiết ý nghĩa trường
1	id	bigint unsigned	Primary Key, Auto Increment	Mã định danh duy nhất của hóa đơn đơn đặt xe.
2	user_id	bigint unsigned	Foreign Key	Khóa ngoại tham chiếu đến bảng người dùng users.

3	car_id	bigint unsigned	Foreign Key	Khóa ngoại tham chiếu đến bảng phương tiện cars.
4	driver_name	varchar(255)	Not Null	Họ và tên của người sẽ trực tiếp điều khiển xe.
5	phone	varchar(50)	Not Null	Số điện thoại liên lạc khẩn cấp khi giao nhận xe.
6	check_in_date	date	Not Null	Mốc ngày giờ khách hàng đăng ký nhận phương tiện.
7	check_out_date	date	Not Null	Mốc ngày giờ khách hàng bắt buộc phải hoàn trả xe.
8	total_price	decimal(15,2)	Not Null	Tổng chi phí thực tế của đơn hàng sau tính toán.
9	status	varchar(50)	Default: 'pending'	Trạng thái xử lý đơn ('pending', 'approved', 'cancelled').
10	created_at	timestamp	Nullable	Thời điểm đơn hàng được gửi lên hệ thống thành công.

11	updated_at	timestamp	Nullable	Thời điểm Admin cập nhật thay đổi trạng thái đơn.
----	------------	-----------	----------	---

Bảng 2.4: Bảng dữ liệu Booking

2.2.5. Bảng contacts (Quản lý tin nhắn liên hệ hỗ trợ từ khách hàng)

Bảng tiếp nhận thông tin yêu cầu gọi lại tư vấn từ phía giao diện Client gửi lên.

STT	Tên trường (Field)	Kiểu dữ liệu (Type)	Ràng buộc (Constraints)	Mô tả chi tiết ý nghĩa trường
1	id	bigint unsigned	Primary Key, Auto Increment	Mã định danh duy nhất của tin nhắn liên hệ hỗ trợ.
2	name	varchar(255)	Not Null	Họ và tên của khách hàng để lại thông tin.
3	email	varchar(255)	Not Null	Địa chỉ Email liên lạc của người cần hỗ trợ.
4	phone	varchar(50)	Not Null	Số điện thoại để nhân viên tổng đài gọi điện trực tiếp.
5	message	text	Not Null	Nội dung chi tiết câu hỏi, thắc mắc cần tư vấn.
6	created_at	timestamp	Nullable	Thời điểm form liên hệ được gửi đi thành công.

7	updated_at	timestamp	Nullable	Thời điểm thông tin liên hệ được xử lý ghi chú.
---	------------	-----------	----------	---

Bảng 2.5: Bảng dữ liệu Contacts

3. THIẾT KẾ CÁCH XỬ LÝ

3.1. Tiến trình Đăng ký tài khoản thành viên

- Mục đích: Khởi tạo thực thể người dùng mới sạch lỗi, bảo mật thông tin an toàn.
- Thuật toán xử lý tuần tự tại Backend:
 1. Tiếp nhận gói dữ liệu HTTP POST Request từ form đăng ký của Client gửi về hệ thống.
 2. Kích hoạt bộ kiểm tra điều kiện dữ liệu (Validator):
 - Kiểm tra trường name và email không được để trống (bắt buộc nhập).
 - Quét CSDL kiểm tra tính duy nhất qua mệnh đề unique:users đối với trường email.
 - Xác thực độ an toàn của password (tối thiểu 8 ký tự) và khớp hoàn toàn với trường password_confirmation.
 3. Nhánh ngoại lệ (Validation Fail): Nếu dữ liệu vi phạm quy tắc, Controller lập tức chặn tiến trình, đóng gói mảng thông báo lỗi và quay lui giao diện (Redirect Back) kèm giữ lại các ô nhập liệu cũ (withInput).
 4. Nhánh luồng chính (Validation Pass): Hệ thống gọi hàm mã hóa Hash::make() để băm mật khẩu thô bằng thuật toán Bcrypt.
 5. Thực thi lệnh Eloquent ORM: User::create() để đẩy bản ghi mới vào MySQL với giá trị mặc định được định cấu hình tự động là role = 'user'.
 6. Gọi hàm Auth::login() để cấp quyền Session đăng nhập lập tức cho người dùng, tái tạo lại mã mã hóa bảo mật token của session trình duyệt (\$request->session()->regenerate()).

7. Điều hướng phản hồi về địa chỉ trang chủ /dashboard kèm theo thông báo chào mừng thành viên mới.

3.2. Tiến trình Xác thực Đăng nhập (Mật khẩu & Google OAuth 2.0)

- Mục đích: Định danh người dùng bảo mật cao, điều phối đúng phân hệ giao diện dựa trên quyền hạn.
- Thuật toán xử lý tuần tự tại Backend:
 - o Luồng xử lý 1: Đăng nhập bằng tài khoản nội bộ (Hệ thống)
 1. Tiếp nhận Email và Mật khẩu từ Client gửi lên.
 2. Kích hoạt hàm kiểm tra xác thực lỗi hệ thống: `Auth::attempt(['email' => $email, 'password' => $password], $remember)`.
 3. Laravel thực hiện tìm kiếm bản ghi theo Email, trích xuất chuỗi mã hóa mật khẩu trong DB và chạy giải mã đối chiếu logic đối với mật khẩu người dùng vừa nhập vào.
 4. Nếu thất bại: Hệ thống tăng biến đếm số lần đăng nhập sai (để chống tấn công Brute-force) và trả lỗi về màn hình Client.
 5. Nếu thành công: Hệ thống làm sạch bộ nhớ đệm session cũ, nạp Token phiên mới. Tiến hành đọc dữ liệu tại cột role trong bảng users:
 - Nếu role == 'admin', điều hướng URL sang phân hệ quản trị cấp cao /admin/dashboard.
 - Nếu role == 'user', điều hướng URL về phân hệ khách hàng /dashboard.
 - o Luồng xử lý 2: Đăng nhập nhanh qua API Google Socialite
 1. Khách hàng bấm nút Google, hệ thống gọi `GoogleAuthController@redirect` để chuyển hướng trình duyệt sang cổng bảo mật của Google API.
 2. Sau khi xác thực danh tính Gmail thành công, Google trả về một chuỗi mã hóa Token an toàn đến đường dẫn Route Callback cấu hình trên Railway.

3. Hàm `GoogleAuthController@callback` tiếp nhận Token, bóc tách lấy ra mảng dữ liệu gồm: id (Mã định danh Google), email và name.
4. Thực hiện câu lệnh truy vấn lồng kiểm tra: `User::where('google_id', $googleUser->id)->first()`.
5. Trường hợp đã tồn tại mã: Đăng nhập thẳng user vào hệ thống và điều hướng về trang chủ Client.
6. Trường hợp email đã đăng ký từ trước: Hệ thống tự động ghi đè mã `google_id` mới vào cột dữ liệu của user đó, thực hiện lưu thay đổi (`$user->save()`) và đăng nhập đồng bộ.
7. Trường hợp khách hàng mới hoàn toàn: Hệ thống tự động chạy lệnh khởi tạo một thực thể User sạch với tên, email lấy từ Google, gán một chuỗi mật khẩu ngẫu nhiên được mã hóa bảo mật cao, thiết lập role = 'user', lưu vào CSDL và cấp quyền Session truy cập lập tức.

3.3. Tiến trình Đăng xuất tài khoản an toàn

- Mục đích: Xóa bỏ hoàn toàn dấu vết phiên làm việc trên trình duyệt, ngăn chặn đánh cắp Session.
- Thuật toán xử lý tuần tự tại Backend:
 1. Tiếp nhận yêu cầu click nút "Đăng xuất" từ giao diện người dùng.
 2. Gọi hàm lỗi xác thực thu hồi quyền: `Auth::guard('web')->logout()`.
 3. Thực thi lệnh xóa bỏ vĩnh viễn và làm rỗng toàn bộ dữ liệu, biến mảng đang lưu trữ tạm thời trong Session của Client: `$request->session()->invalidate()`.
 4. Thực hiện làm mới lại chuỗi ký tự Token chống giả mạo của form để bảo mật trình duyệt cho phiên truy cập tiếp theo: `$request->session()->regenerateToken()`.
 5. Phát ra phản hồi điều hướng trình duyệt quay trở lại giao diện trang đăng nhập hệ thống công khai.

3.4. Tiến trình Quản lý và cập nhật hồ sơ cá nhân (Profile)

- Mục đích: Thay đổi thông số tài khoản và cung cấp module hủy bỏ tài khoản theo yêu cầu bảo mật.
- Thuật toán xử lý tuần tự tại Backend:
 - o Luồng Cập nhật thông tin:
 1. Hàm ProfileController@edit thực hiện truy vấn Eloquent lấy thông tin trực tiếp từ ID của Session hiện tại để điền vào Form.
 2. Người dùng sửa thông tin và ấn nút "Lưu". Controller chạy bộ quy tắc validate dữ liệu Họ tên (bắt buộc), Email (bắt buộc, đúng định dạng, loại trừ chính ID hiện tại để không báo trùng lặp lỗi).
 3. Dữ liệu chuẩn, hệ thống thực hiện gán lại giá trị cho Model và gọi hàm \$request->user()->save() để đồng bộ hóa xuống MySQL.
 - o Luồng Xóa bỏ tài khoản thành viên:
 1. Khách hàng nhấn chọn nút "Xóa tài khoản" tại giao diện Profile.
 2. Hệ thống bắt buộc hiển thị trường nhập mật khẩu hiện tại để xác minh chính chủ an toàn.
 3. Xác thực mật khẩu chính xác, Controller gọi câu lệnh xóa vĩnh viễn thực thể: \$user->delete().
 4. Hệ thống tự động thu hồi Session, đăng xuất tài khoản và xóa bản ghi khỏi bảng users.

3.5. Tiến trình Xem và lọc danh sách xe theo phân khúc danh mục

- Mục đích: Trích xuất chính xác kho xe hiển thị và tối ưu hóa tốc độ tải trang Front-end.
- Thuật toán xử lý tuần tự tại Backend:
 1. Khi khách hàng truy cập route /dashboard, hàm HomeController@index được kích hoạt gọi lệnh lấy dữ liệu.
 2. Hệ thống thực hiện song song hai truy vấn Eloquent ORM:
 - Lấy toàn bộ danh mục phân khúc: Category::all() để render lên thanh Menu Tab lọc.

- Lấy danh sách xe hoạt động: `Car::query()`.
3. Hệ thống kiểm tra tham số lọc trên URL (ví dụ: `?category=3` khi khách click tab xe 7 chỗ SUV):
 - Nếu có tham số lọc: Thêm mệnh đề điều kiện SQL: -
`>where('category_id', $request->category)`.
 - Nếu không có tham số: Mặc định lấy toàn bộ danh sách các xe có trạng thái khả dụng kinh doanh.
 4. Gắn thêm bộ lọc bảo mật loại bỏ các xe đang ở trạng thái ẩn hoặc ngưng phục vụ.
 5. Đóng gói mảng dữ liệu bộ sưu tập các xe (`$cars`) truyền sang tầng hiển thị Blade để render cấu trúc dạng lưới (Grid Card) ra màn hình Client.

3.6. Tiến trình Xem chi tiết thông số kỹ thuật xe

- Mục đích: Truy vấn thuộc tính chi tiết phương tiện an toàn, xử lý triệt để ngoại lệ sai ID.
- Thuật toán xử lý tuần tự tại Backend:
 1. Khách hàng click vào thẻ xe, trình duyệt gửi yêu cầu GET Request kèm mã ID xe lên địa chỉ route `/xe/{id}`.
 2. Hàm xử lý `HomeController@show($id)` tiếp nhận chuỗi tham số ID từ URL.
 3. Thực thi câu lệnh truy vấn bắt lỗi nghiêm ngặt: `Car::with('category')->findOrFail($id)`.
 4. Cơ chế bắt lỗi hệ thống: Hàm `findOrFail` sẽ tự động thực hiện tìm kiếm ID trong bảng `cars`. Nếu người dùng cố tình thay đổi ID trên thanh địa chỉ thành một số không tồn tại, hệ thống lập tức bắt lấy ngoại lệ `ModelNotFoundException` và tự động điều hướng sang trang giao diện báo lỗi an toàn 404 - Phương tiện không tồn tại thay vì sập mã nguồn.
 5. Nếu ID chuẩn xác, CSDL trả về bản ghi phương tiện, Controller bóc tách toàn bộ mảng thuộc tính: Nhiên liệu, Hộp số, Số chỗ, Giá thuê đồ trực quan lên giao diện chi tiết.

3.7. Tiến trình Đặt đơn thuê xe & Xử lý hóa đơn tạm tính (Mô phỏng Checkout)

- Mục đích: Ghi nhận lịch trình thuê xe, tính toán chi phí chính xác và chuyển đơn hàng vào trạng thái chờ duyệt.
- Thuật toán xử lý tuần tự tại Backend:
 1. Khách hàng chọn xe, nhập thông tin Họ tên người lái, Số điện thoại, chọn Ngày nhận xe (check_in_date) và Ngày trả xe (check_out_date) tại trang thanh toán.
 2. Khách hàng nhấn nút Xác nhận, hệ thống gửi gói dữ liệu qua phương thức POST Request về route /dat-xe.
 3. Hàm xử lý BookingController@store kích hoạt bộ quy tắc Validate nghiêm ngặt: Ngày nhận và ngày trả bắt buộc phải đúng định dạng date, ngày trả bắt buộc phải sau ngày nhận xe.
 4. Controller tiến hành xử lý thuật toán thời gian: Chuyển đổi chuỗi ngày nhận/trả thành đối tượng thời gian hệ thống, tính toán khoảng cách hiệu số tổng số ngày thuê (\$totalDays).
 5. Truy vấn dữ liệu để lấy đơn giá ngày của chiếc xe đó từ bảng cars. Thực hiện thuật toán tính tổng chi phí hóa đơn: $\$totalPrice = \$totalDays * \$car->price_per_day$.
 6. Khởi chạy tiến trình ghi dữ liệu bảo mật, gán mã ID của tài khoản đang đăng nhập thông qua hàm Auth::id().
 7. Thực thi câu lệnh INSERT tạo bản ghi mới vào bảng bookings với cấu hình mặc định ban đầu tại cột trường trạng thái là status = 'pending' (Chờ duyệt).
 8. Trả về thông báo Pop-up đặt xe thành công và điều hướng khách hàng về trang lịch sử đơn.

3.8. Tiến trình Xem lịch sử đơn đặt xe cá nhân của khách hàng

- Mục đích: Trích xuất riêng tư thông tin lịch sử giao dịch của riêng tài khoản đang đăng nhập.
- Thuật toán xử lý tuần tự tại Backend:
 1. Khách hàng truy cập vào phân hệ "Lịch sử đặt đơn" tại Client.

2. Hàm xử lý `BookingController@history` lấy ID định danh duy nhất của tài khoản từ phiên làm việc Session hiện tại qua mã: `$userId = Auth::id()`.
3. Thực thi câu lệnh truy vấn Eloquent ORM có điều kiện lọc và liên kết bảng tăng tốc (Eager Loading): `Booking::where('user_id', $userId)->with('car')->orderBy('created_at', 'desc')->get()`.
4. Hệ thống kiểm tra dữ liệu mảng trả về: Nếu mảng rỗng, giao diện hiển thị thông báo "Bạn chưa có giao dịch đặt xe nào". Nếu có dữ liệu, bóc tách danh sách đơn đổ lên bảng hiển thị trực quan cho khách hàng theo dõi tiến độ phê duyệt đơn.

3.9. Tiến trình Tiếp nhận thông tin liên hệ hỗ trợ từ khách hàng (Client Contact)

- Mục đích: Thu thập dữ liệu lời nhắn và số điện thoại yêu cầu gọi lại tư vấn từ phía Client.
- Thuật toán xử lý tuần tự tại Backend:
 1. Khách hàng nhập Form tại trang Liên hệ (/lien-he) gồm: Họ tên, Email, Số điện thoại và Nội dung tin nhắn cần hỗ trợ.
 2. Nhấn nút Gửi, form đóng gói dữ liệu truyền tải theo chuẩn POST Request về route `/contact/submit`.
 3. Hàm `ContactController@store` tiếp nhận dữ liệu, thực hiện kiểm tra validate đầu vào: Trường Số điện thoại và trường nội dung lời nhắn bắt buộc phải điền thông tin chi tiết.
 4. Nếu dữ liệu hợp lệ, Controller gọi lệnh thực thi lỗi: `Contact::create($request->all())`.
 5. Laravel Eloquent chuyển đổi dữ liệu thành câu lệnh `INSERT INTO contacts...` đẩy bản ghi tin nhắn mới trực tiếp vào cơ sở dữ liệu trên đám mây Railway.
 6. Hệ thống thực hiện lệnh điều hướng quay trở lại kèm thông báo đẩy Toast xanh lá: "CARENTO đã ghi nhận thông tin liên hệ của bạn. Nhân viên tư vấn sẽ chủ động gọi điện lại hỗ trợ trong giây lát!".

3.10. Tiến trình Thống kê chỉ số báo cáo tại Trung tâm điều hành (Admin Dashboard)

- Mục đích: Tự động hóa khâu tổng hợp dữ liệu tài chính, hoạt động kinh doanh hiển thị thời gian thực cho Admin.
- Thuật toán xử lý tuần tự tại Backend:
 1. Khi tài khoản có đặc quyền Admin truy cập route /admin/dashboard, hệ thống kích hoạt hàm điều khiển DashboardController@index.
 2. Hệ thống thực hiện đồng loạt các câu lệnh truy vấn tổng hợp (Aggregation Queries) sâu dưới Database MySQL bao gồm:
 - Tính tổng doanh thu thực tế từ các đơn thuê xe đã hoàn thành phê duyệt bằng lệnh: `Booking::where('status', 'approved')->sum('total_price')`.
 - Đếm tổng số lượng đơn đặt xe trên hệ thống: `Booking::count()`.
 - Đếm số lượng đơn hàng mới đang nằm trong hàng đợi chờ xử lý gấp: `Booking::where('status', 'pending')->count()`.
 - Đếm tổng số lượng thành viên khách hàng đã đăng ký: `User::where('role', 'user')->count()`.
 3. Hệ thống gom toàn bộ các biến số liệu tính toán được vào một mảng tài nguyên chung và đẩy sang giao diện Admin Dashboard để render trực quan lên các khối đồ họa Grid.

3.11. Tiến trình Quản lý kho xe và Đổi trạng thái xe nhanh (Admin CRUD Cars)

- Mục đích: Điều phối kho tài sản xe ô tô kinh doanh và hỗ trợ chuyển đổi trạng thái bảo dưỡng nhanh trên Grid.
- Thuật toán xử lý tuần tự tại Backend:
 - o Luồng xử lý CRUD truyền thống:
 - Khi Admin thêm/sửa xe, hệ thống bắt nhận tệp tin hình ảnh xe tải lên, mã hóa lại tên tệp tin để tránh trùng lặp, lưu vào thư mục asset, đồng thời thực hiện câu lệnh INSERT/UPDATE dữ liệu vào bảng cars dựa theo liên kết khóa ngoại category_id chuẩn xác. Khi

bấm xóa (Delete), hệ thống gọi lệnh DELETE xóa bản ghi ra khỏi Database.

- Luồng xử lý Thay đổi trạng thái vận hành xe nhanh (PATCH status):
 1. Tại bảng lưới Grid danh sách quản lý xe, Admin click chuột vào nút gạt/nút bấm chuyển đổi nhanh trạng thái của chiếc xe.
 2. Trình duyệt gửi một yêu cầu chỉnh sửa một phần PATCH Request gửi trực tiếp về địa chỉ route `/admin/cars/{id}/status`.
 3. Hàm điều khiển `CarController@updateStatus` tiếp nhận ID xe, thực hiện câu lệnh tìm kiếm chiếc xe trong bảng dữ liệu: `$car = Car::findOrFail($id)`.
 4. Controller thực hiện thuật toán đảo ngược logic trạng thái hiện tại của phương tiện: Nếu trạng thái hiện tại là 'available' (Sẵn sàng phục vụ), hệ thống tự động gán đổi thành 'maintenance' (Đưa xe vào xưởng bảo dưỡng định kỳ) và ngược lại.
 5. Gọi câu lệnh lưu thay đổi: `$car->save()`. Hệ thống cập nhật màu sắc cờ hiệu trạng thái xe trên màn hình Admin ngay lập tức mà không cần tải lại trang web.

3.12. Tiến trình Quản lý và kiểm soát danh sách thành viên khách hàng

- Mục đích: Trích xuất tệp hồ sơ người dùng và thống kê mức độ tương tác đơn đặt xe của từng người.
- Thuật toán xử lý tuần tự tại Backend:
 1. Admin nhấn chọn mục Quản lý khách hàng tại đường dẫn route `/admin/customers`.
 2. Hàm xử lý `CustomerController@index` được kích hoạt gọi câu lệnh truy vấn SQL nâng cao: `User::where('role', 'user')->withCount('bookings')->orderBy('created_at', 'desc')->get()`.
 3. Cơ chế câu lệnh tối ưu: Mệnh đề `withCount('bookings')` ra lệnh cho MySQL thực hiện đếm số lượng bản ghi hóa đơn tương ứng của từng khách hàng từ bảng `bookings` kết nối qua khóa ngoại ID, giúp giảm thiểu số lượng

câu lệnh truy vấn xuống Database (Tránh lỗi thất nút cổ chai hiệu năng N+1 query).

4. Hệ thống gọi nạp tệp cấu hình thiết kế định dạng giao diện tĩnh dành riêng cho trang khách hàng (customers.css) để render bảng danh sách hiển thị mượt mà lên màn hình Admin.

3.13. Tiến trình Điều phối Phê duyệt / Hủy đơn đặt xe (Admin Bookings)

- Mục đích: Điều phối trạng thái vòng đời đơn hàng và đồng bộ tự động trạng thái khóa xe trong kho dữ liệu.
- Thuật toán xử lý tuần tự tại Backend:
 1. Admin truy cập bảng danh sách đơn đặt xe (/admin/bookings), xem chi tiết nội dung đơn hàng và click chọn nút hành động điều phối.
 2. Trình duyệt đóng gói tham số hành động và ID đơn gửi qua phương thức PATCH Request về route xử lý trạng thái.
 3. Hàm AdminBookingController@updateStatus tìm đơn đặt xe bằng lệnh Booking::findOrFail(\$id) và rà soát lệnh điều phối của Admin:
 - Trường hợp 1 (Admin chọn nút Phê duyệt - Approve): Hệ thống cập nhật trường trạng thái đơn thành status = 'approved'. Đồng thời, kích hoạt câu lệnh cập nhật đồng bộ sang bảng kho xe, tự động chuyển đổi trạng thái của chiếc xe nằm trong đơn hàng đó sang mã trạng thái khóa phương tiện status = 'rented', ngăn chặn tuyệt đối tình trạng hệ thống cho phép khách hàng khác đặt trùng lịch chiếc xe này.
 - Trường hợp 2 (Admin chọn nút Từ chối/Hủy đơn - Cancel): Hệ thống cập nhật trường trạng thái hóa đơn thành status = 'cancelled', đưa chi phí đơn về diện không tính toán doanh thu và giải phóng chiếc xe về trạng thái sẵn sàng phục vụ ban đầu.
 4. Gọi lệnh thực thi lưu thay đổi, hệ thống tự động gọi hàm tính toán làm mới lại biểu đồ và các con số chỉ số tài chính tổng doanh thu thực tế ngoài Dashboard Admin thời gian thực.

3.14. Tiến trình Tiếp nhận và xử lý Tin nhắn liên hệ của khách hàng (Admin Messages)

- Mục đích: Trích xuất danh sách số điện thoại yêu cầu hỗ trợ hiển thị lên không gian làm việc chuyên biệt để nhân viên nhắc máy gọi điện trực tiếp làm việc với khách.
- Thuật toán xử lý tuần tự tại Backend:
 1. Admin hoặc nhân viên trực tổng đài nhà xe bấm chọn mục "Quản lý tin nhắn" tại địa chỉ route /admin/messages.
 2. Hàm điều khiển MessageController@index thực hiện câu lệnh truy vấn dữ liệu từ bảng tin nhắn liên hệ: `Contact::orderBy('created_at', 'desc')->get()`.
 3. Hệ thống tiến hành nạp cấu trúc định dạng đồ họa từ tệp phong cách thiết kế riêng messages.css để phân chia giao diện thành phân hệ không gian làm việc chuyên nghiệp (gồm Sidebar danh sách tin nhắn mới bên trái và màn hình hiển thị nội dung chi tiết lời nhắn to rõ ràng ở trung tâm).
 4. Giao diện bóc tách hiển thị rõ ràng thông tin: Tên khách hàng, Email, Số điện thoại liên lạc khẩn cấp và nội dung chi tiết lời nhắn yêu cầu tư vấn của khách.
 5. Tiến trình nghiệp vụ ngoại tuyến thực tế (Xử lý Offline): Nhân viên trực tổng đài nhìn trực quan vào số điện thoại hiển thị trên màn hình Workspace này, tiến hành sử dụng điện thoại trực tiếp bấm số gọi điện lại làm việc với khách hàng để giải đáp thắc mắc thủ tục, hẹn lịch nhận xe và hoàn tất chốt hợp đồng thuê xe offline trực tiếp với khách hàng.

4. KẾT QUẢ GIAO DIỆN WEB

Phân hệ giao diện của hệ thống CARENTO được thiết kế theo ngôn ngữ phẳng hiện đại, ứng dụng bộ thư viện Tailwind CSS tối ưu hóa trải nghiệm người dùng (UI/UX). Giao diện sử dụng hệ thống phông chữ Inter đồng bộ toàn trang, giúp hiển thị chữ nét căng và sạch lỗi tiếng Việt. Nhờ ứng dụng các kỹ thuật thiết kế Responsive thích ứng đa nền tảng, toàn bộ bố cục, biểu mẫu và bảng biểu đều tự động co giãn tương thích

chuẩn mực trên cả hai môi trường hiển thị: Máy tính cá nhân (Desktop) và Điện thoại di động (Mobile).

4.1. Giao diện cho máy tính

4.1.1. Giao diện Trang đăng ký tài khoản thành viên mới

Tạo tài khoản mới
Đăng ký để bắt đầu hành trình của bạn cùng chúng tôi

Họ và tên
Nguyễn Văn A

Địa chỉ Email
name@example.com

Mật khẩu

Xác nhận mật khẩu

ĐĂNG KÝ TÀI KHOẢN

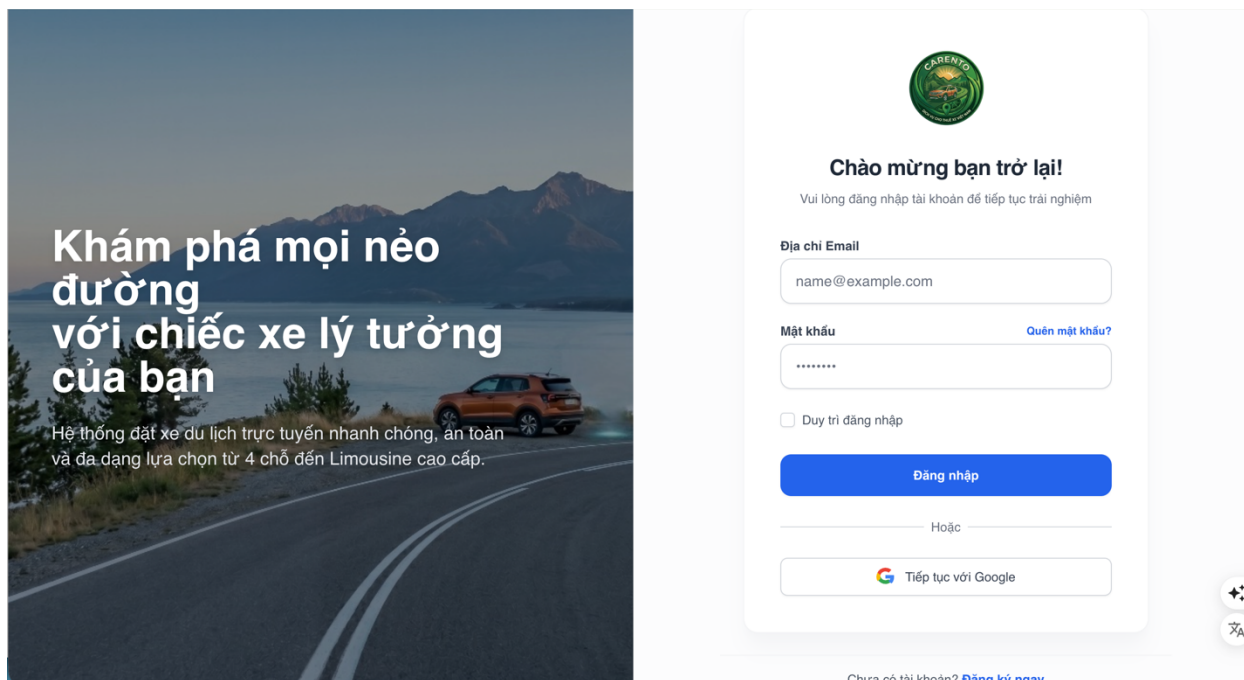
Hoặc

Tiếp tục với Google

Đã có tài khoản? [Đăng nhập ngay](#)

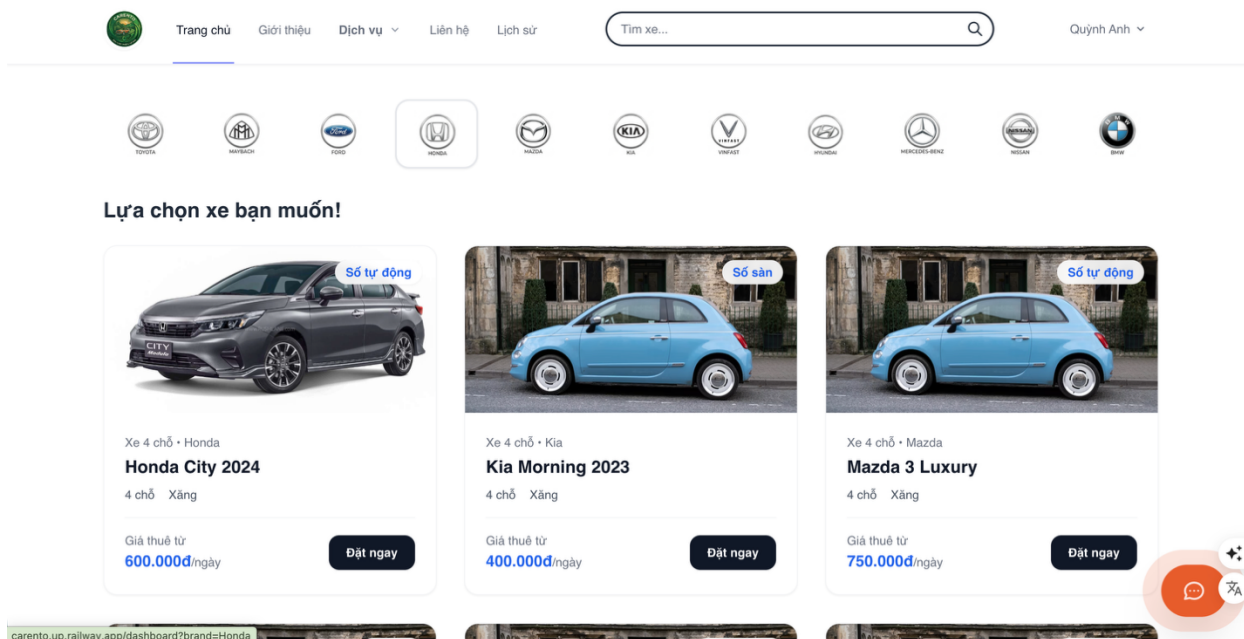
Hình 1: Giao diện Trang đăng ký tài khoản thành viên mới trên máy tính

4.1.2. Giao diện Trang đăng nhập hệ thống (Xác thực kép)



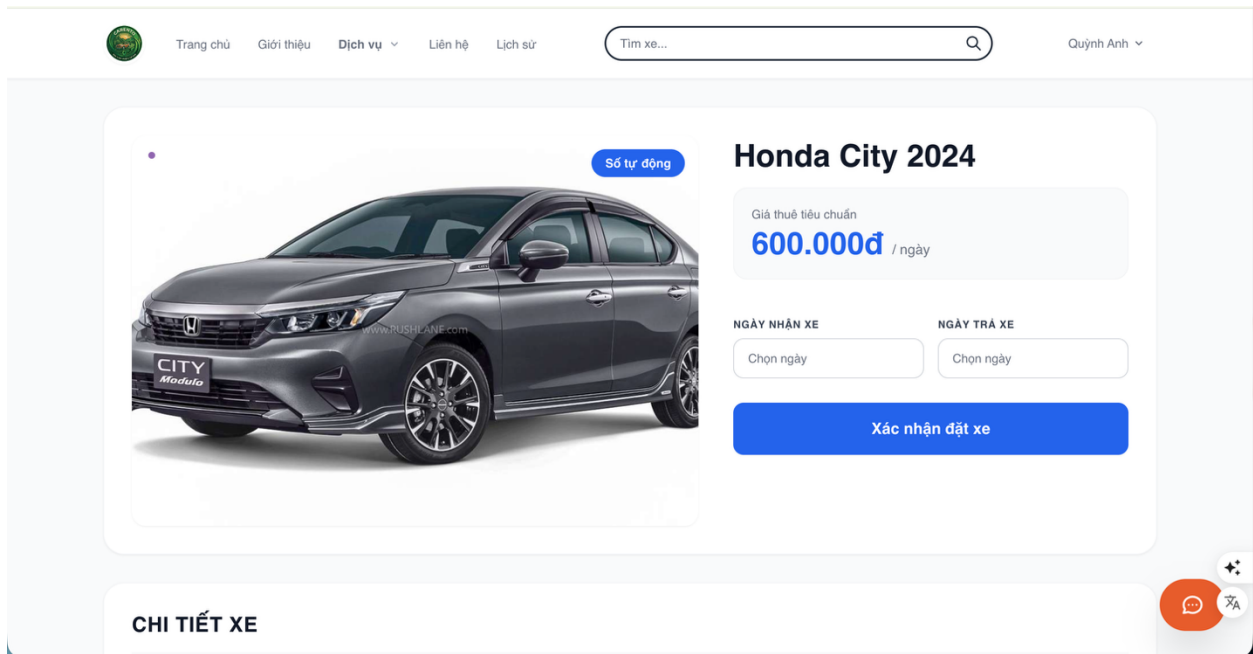
Hình 2: Giao diện Trang đăng nhập tích hợp Google OAuth 2.0 trên máy tính

4.1.3. Giao diện Trang chủ và Bộ lọc phân khúc xe dành cho Khách hàng

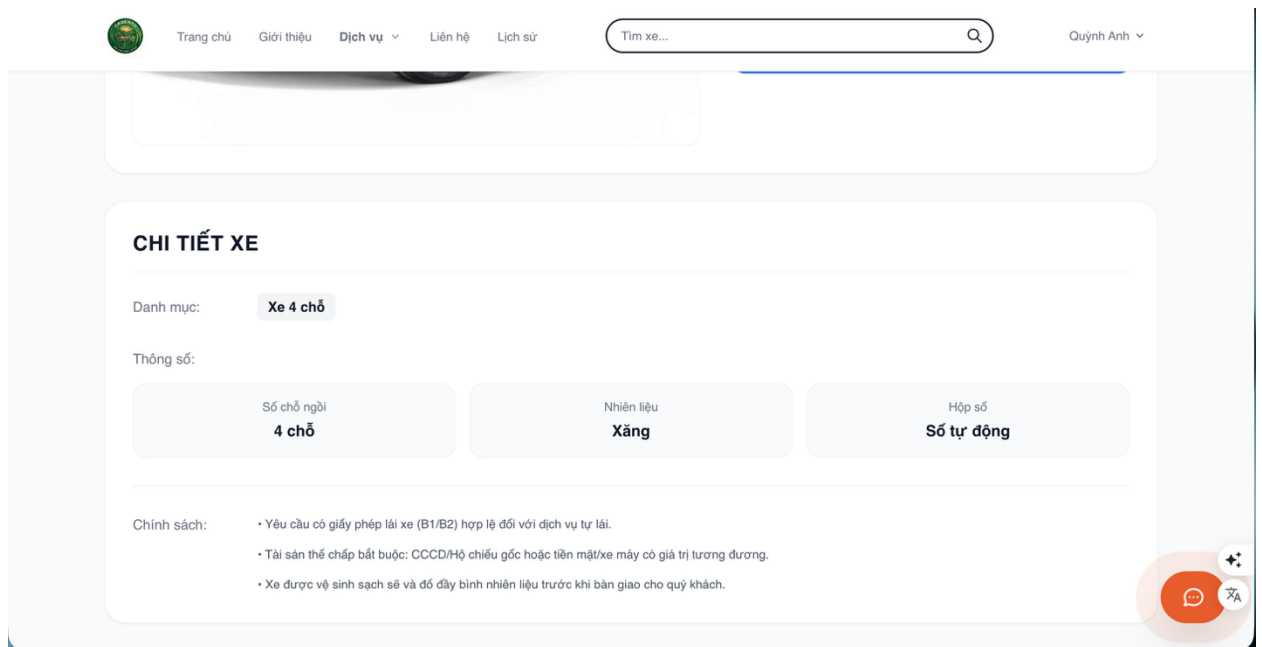


Hình 3: Giao diện Trang chủ và Bộ lọc danh mục phân khúc xe trên máy tính

4.1.4. Giao diện Trang hiển thị thông số kỹ thuật chi tiết xe



Hình 4: Giao diện Trang thông số kỹ thuật chi tiết phương tiện trên máy tính 1



Hình 5: Giao diện Trang thông số kỹ thuật chi tiết phương tiện trên máy tính 2

4.1.5. Giao diện Trang điền thông tin và Xác nhận đặt xe thuê (Checkout)

Trang chủ

Giới thiệu

Dịch vụ

Liên hệ

Lịch sử

Tìm xe...

Quỳnh Anh

THANH TOÁN & XÁC NHẬN

Thông tin người đặt

HỌ VÀ TÊN

Quỳnh Anh

EMAIL XÁC NHẬN

qanh0909@gmail.com

Số điện thoại liên hệ *

Nhập số điện thoại để nhận xe...

☐ Tôi muốn giao xe tận nơi

Chúng tôi sẽ giao xe đến tận nhà bạn (Phụ phí mặc định: +200.000đ).

Hướng dẫn nhận xe tại bãi

Giấy tờ bắt buộc cần mang theo:

Giấy tờ nhân thân (CCCD) hoặc Hộ chiếu bản gốc.

Tóm tắt đơn hàng #5

HONDA

Honda City 2024

Số tự động • 4 chỗ

Ngày nhận xe:

03/06/2026

Ngày trả xe:

05/06/2026

Giá thuê xe:

1.200.000đ

TỔNG THANH TOÁN

1.200.000đ

Phương thức thanh toán

☒ Thanh toán khi nhận xe (COD)

Hình 6: Giao diện Form đặt xe và Hóa đơn tạm tính thời gian thực trên máy tính 1

Trang chủ

Giới thiệu

Dịch vụ

Liên hệ

Lịch sử

Tìm xe...

Quỳnh Anh

Số điện thoại mới ư?

Bạn có muốn lưu số này cho những lần đặt xe tiếp theo không?

☐

Tôi muốn giao xe tận nơi

Chúng tôi sẽ giao xe đến tận nhà bạn (Phụ phí mặc định: +200.000đ).

☒

Địa chỉ nhận xe *

T Mart, Phố Nguyễn Trác, Phường Dương Nội, Hà Nội

☒ Chọn từ bản đồ

Hướng dẫn nhận xe tại bãi

Giấy tờ bắt buộc cần mang theo:

Căn cước công dân (CCCD) hoặc Hộ chiếu bản gốc.

Giấy phép lái xe hợp lệ (hạng B1/B2 trở lên).

Tài sản thế chấp: CCCD/Hộ chiếu bản gốc hoặc tài sản có giá trị tương đương.

Địa điểm bãi xe (nếu không chọn giao tận nơi):

☒ Bãi xe Đại học Phenikaa, Hà Đông, Hà Nội

HONDA

Honda City 2024

Số tự động • 4 chỗ

Ngày nhận xe:

03/06/2026

Ngày trả xe:

05/06/2026

Giá thuê xe:

1.200.000đ

Phụ phí giao xe:

200.000đ

TỔNG THANH TOÁN

1.400.000đ

Phương thức thanh toán

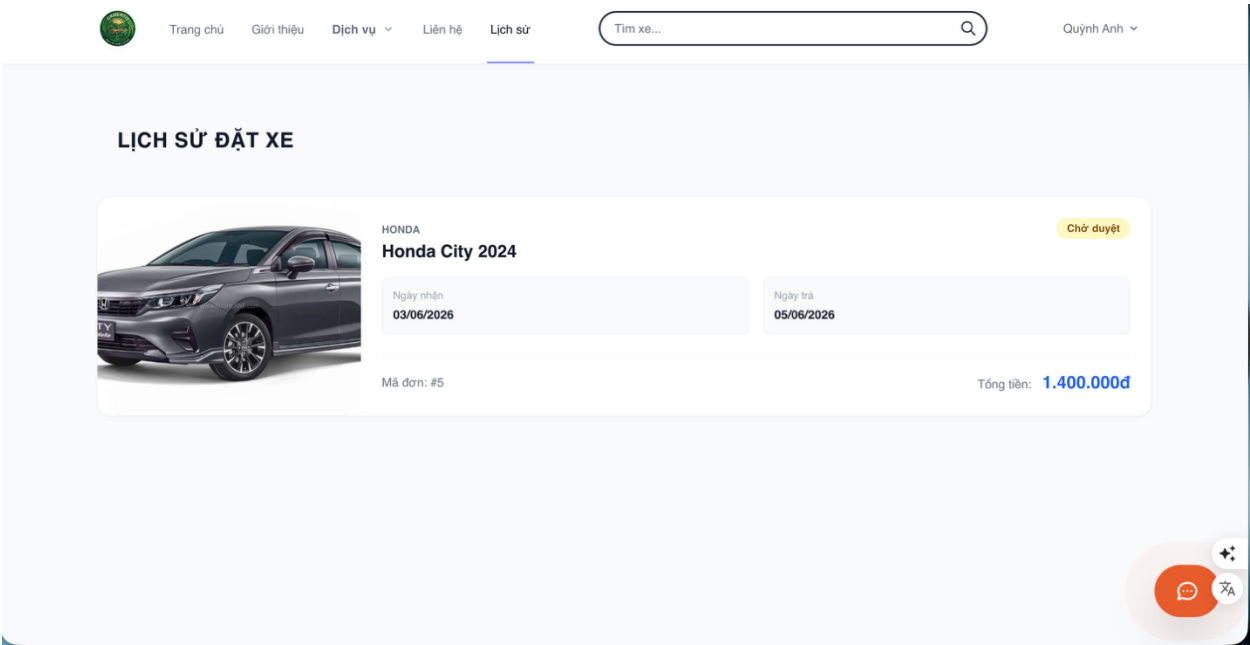
☒ Thanh toán khi nhận xe (COD)

☐ Chuyển khoản VietQR

Xác nhận đặt xe

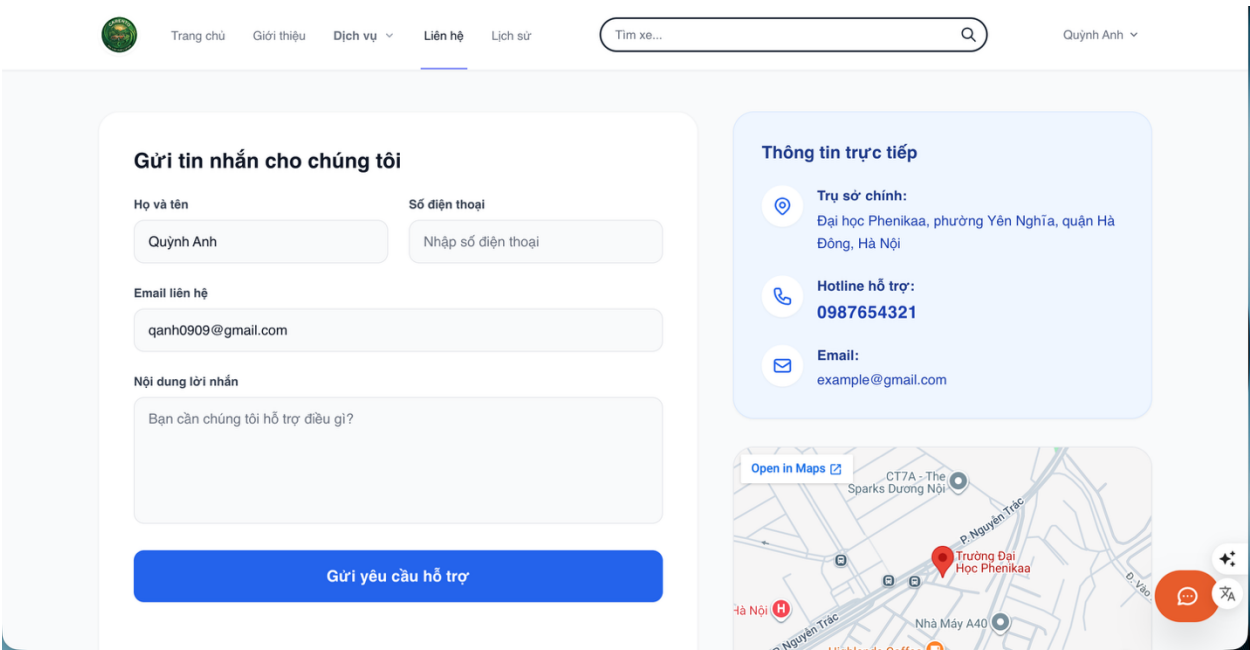
Hình 7: Giao diện Form đặt xe và Hóa đơn tạm tính thời gian thực trên máy tính 2

4.1.6. Giao diện Trang Lịch sử đơn đặt xe cá nhân của Khách hàng



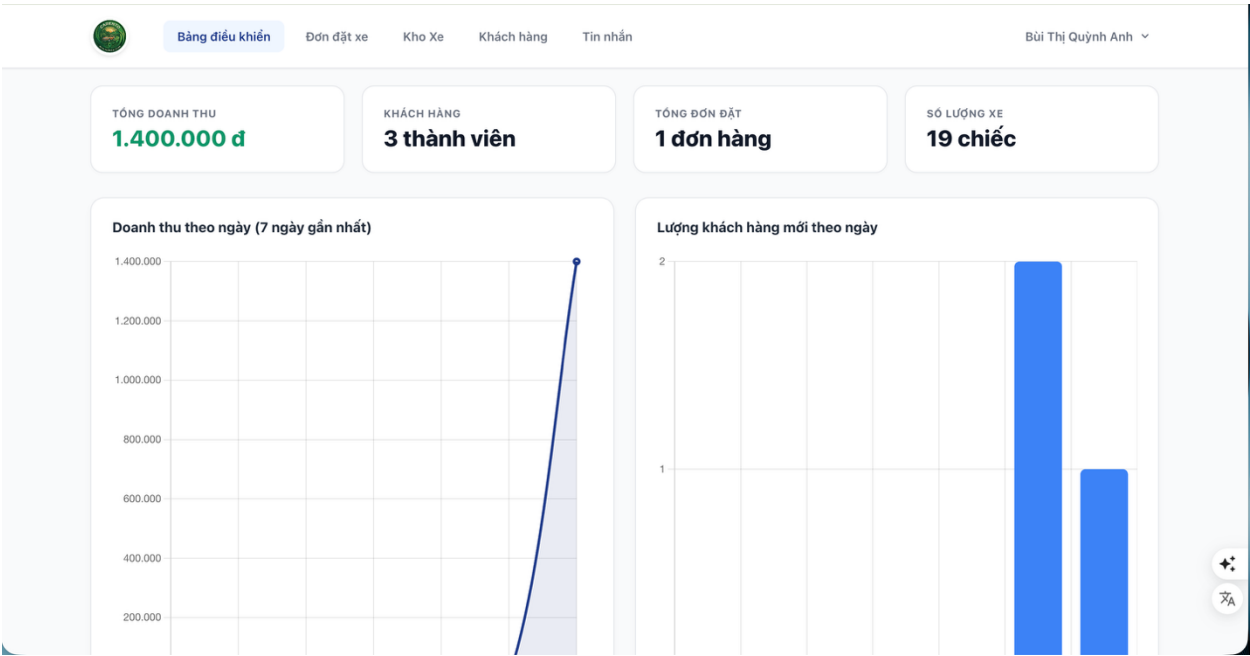
Hình 8: Giao diện Bảng quản lý lịch sử đơn đặt xe của Khách hàng trên máy tính

4.1.7. Giao diện Trang Liên hệ hỗ trợ và Để lại số điện thoại tư vấn



Hình 9: Giao diện Form gửi thông tin liên hệ hỗ trợ tư vấn trên máy tính

4.1.8. Giao diện Trang trung tâm điều hành Admin Dashboard thống kê tài chính



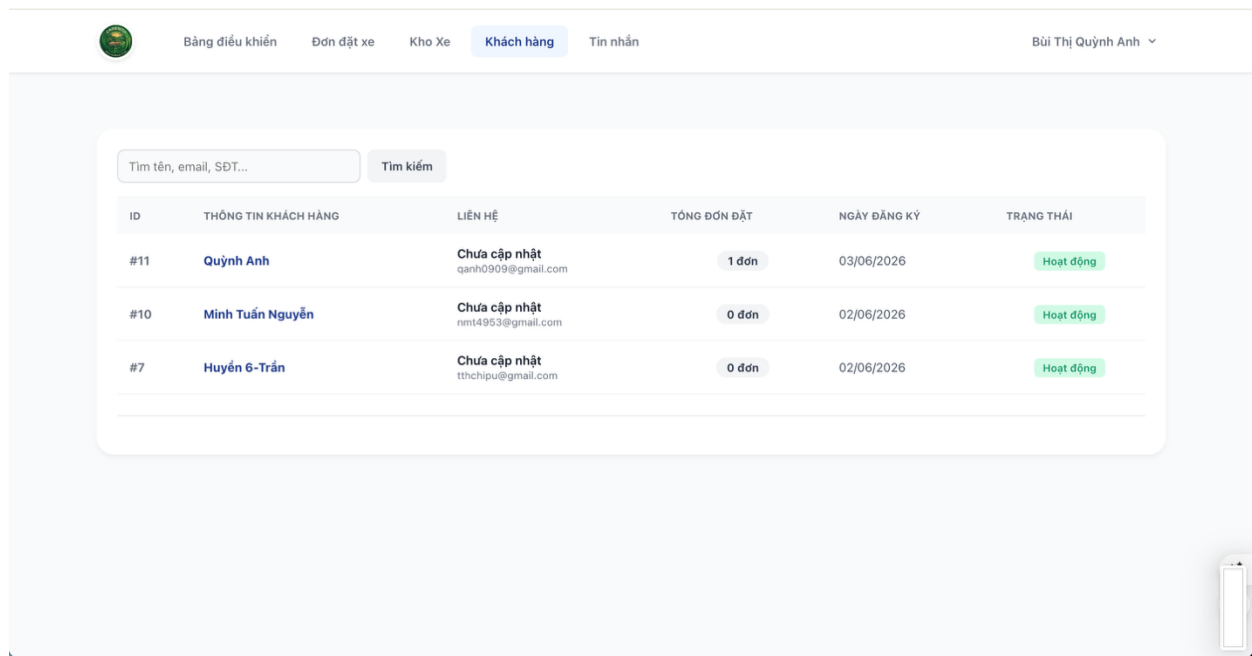
Hình 10: Giao diện Trung tâm điều hành Admin Dashboard thống kê trên máy tính

4.1.9. Giao diện Trang Admin Quản lý kho xe và Đổi trạng thái phương tiện nhanh

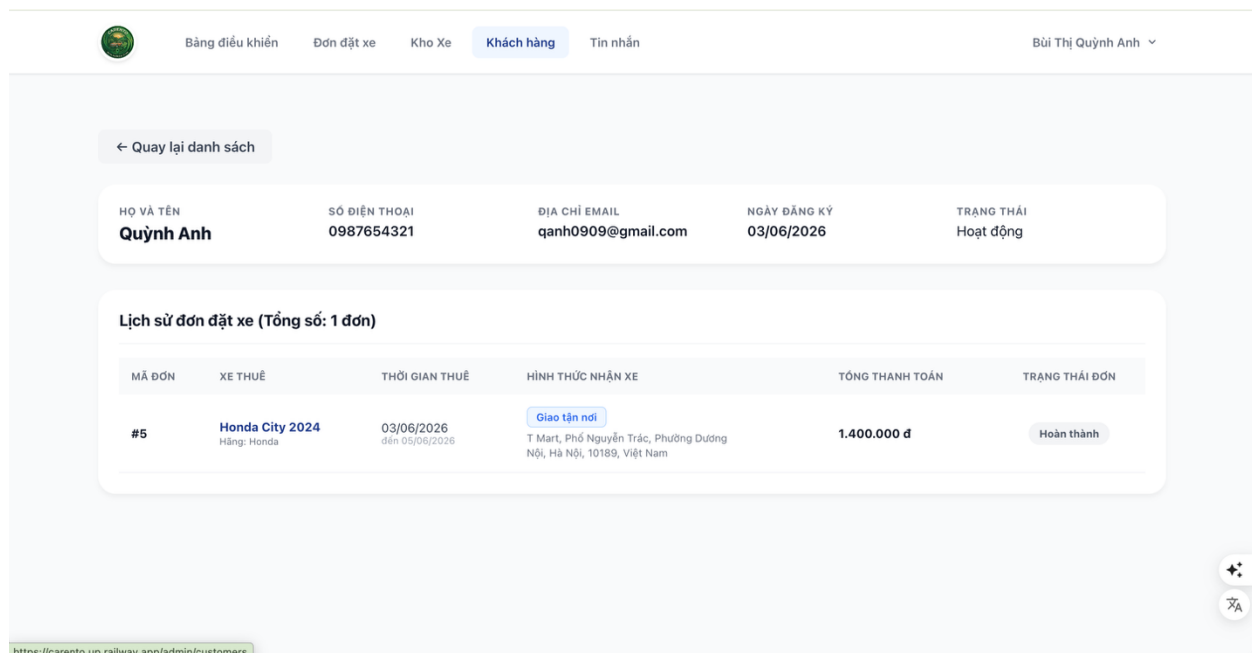
HÌNH ẢNH	TÊN XE & HÃNG	LOẠI / CHỖ NGỒI	GIÁ THUÊ/NGÀY	TRẠNG THÁI	THAO TÁC
No Image	Hyundai Solati Hyundai	Xe 16 chỗ Số sàn	1.500.000 đ	Sẵn sàng	Sửa Xóa
No Image	Hyundai Solati Limousine 11 Chỗ Hyundai	Limousine cao cấp Số tự động	3.000.000 đ	Sẵn sàng	Sửa Xóa
No Image	Kia Carnival Royal Kia	Limousine cao cấp Số tự động	2.000.000 đ	Sẵn sàng	Sửa Xóa
No Image	Ford Transit Limousine 9 Chỗ Ford	Limousine cao cấp Số sàn	2.500.000 đ	Sẵn sàng	Sửa Xóa
No Image	Thaco Bluesky Thaco	Xe 45 chỗ Số sàn	3.200.000 đ	Sẵn sàng	Sửa Xóa
No Image	Hyundai Universe Hyundai	Xe 45 chỗ Số sàn	3.500.000 đ	Sẵn sàng	Sửa Xóa

Hình 11: Giao diện Bảng quản lý kho xe và nút đổi trạng thái nhanh trên máy tính

4.1.10. Giao diện Trang Admin Quản lý danh sách thành viên khách hàng

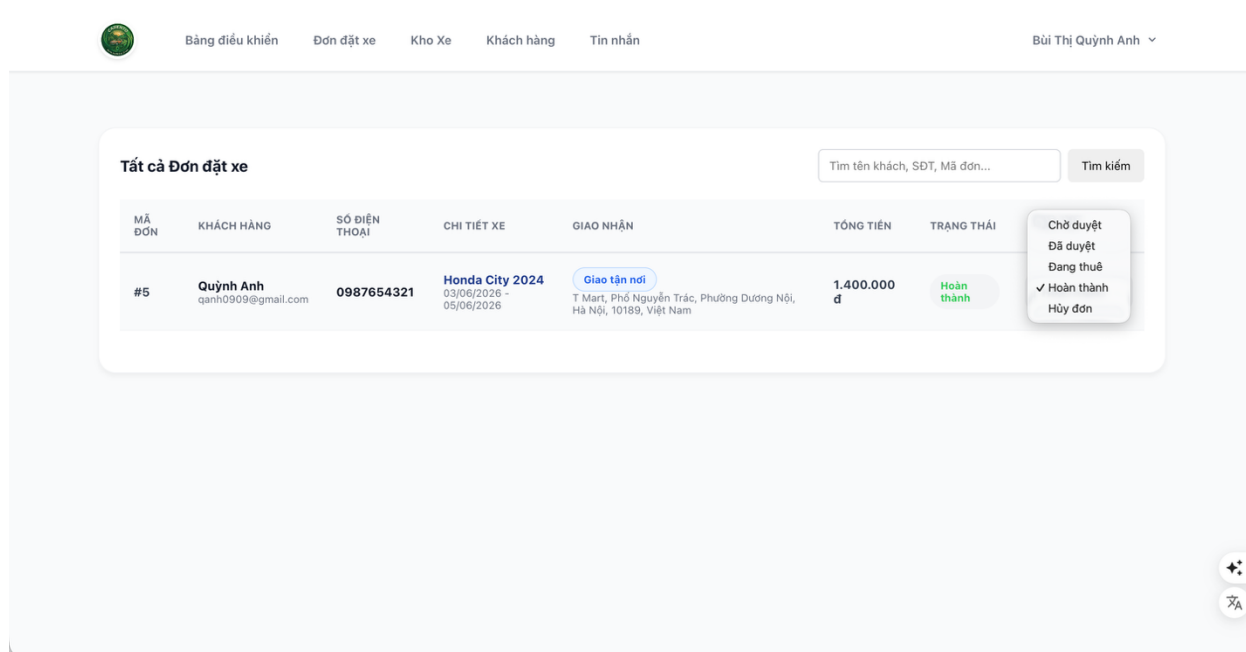


Hình 12: Giao diện Trang Admin quản lý hồ sơ danh sách thành viên trên máy tính 1



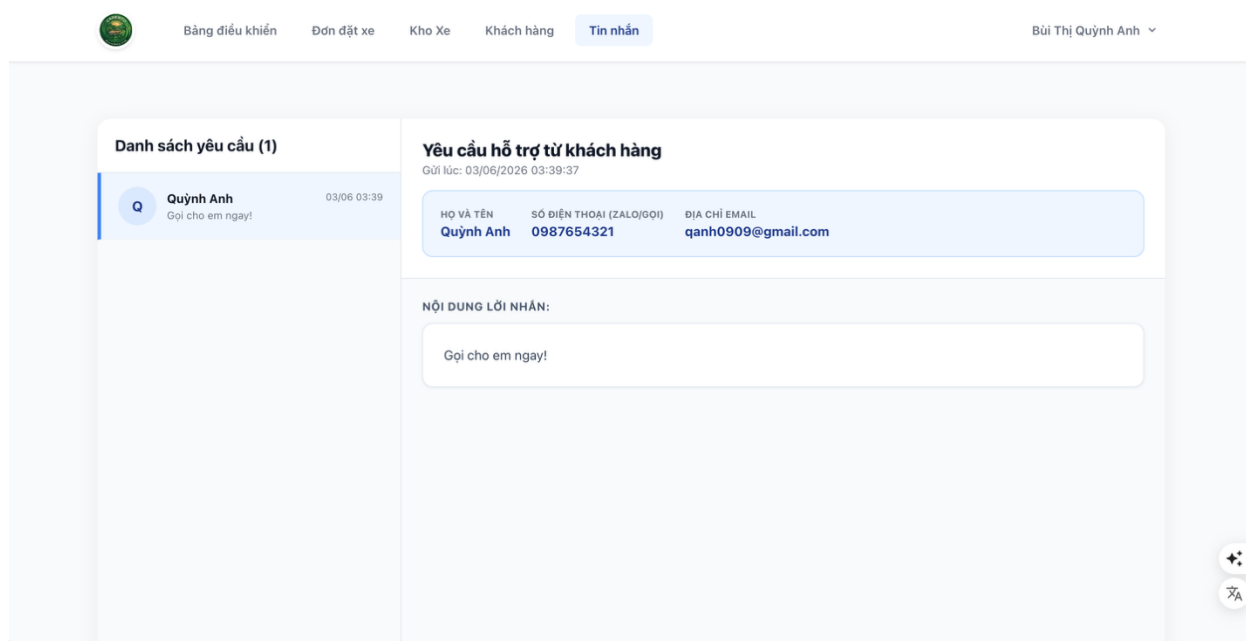
Hình 13: Giao diện Trang Admin quản lý hồ sơ danh sách thành viên trên máy tính 2

4.1.11. Giao diện Trang Admin Điều phối Phê duyệt / Hủy đơn đặt xe



Hình 14: Giao diện Trang Admin điều phối phê duyệt đơn đặt xe trên máy tính

4.1.12. Giao diện Trang Admin Tiếp nhận quản lý tin nhắn liên hệ hỗ trợ



Hình 15: Giao diện Không gian làm việc quản lý tin nhắn liên hệ của Admin trên máy tính

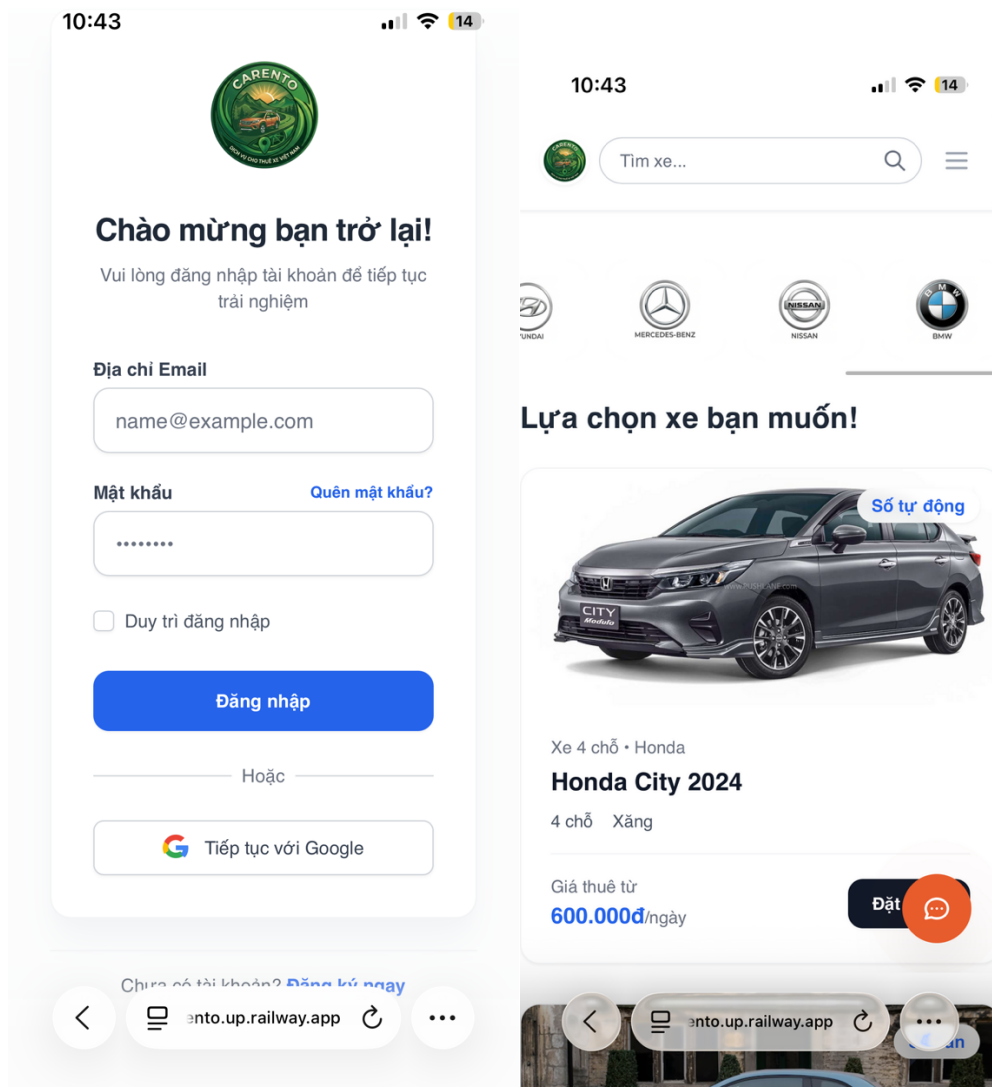
4.2. Giao diện điện thoại

- Nhờ áp dụng triệt để nguyên lý thiết kế ưu tiên thiết bị di động (Mobile-First) và hệ thống tiền tố điểm gãy (Breakpoints) tối ưu của Tailwind CSS (như sm:, md:, lg:), toàn bộ cấu trúc giao diện đồ họa phức tạp trên máy tính của ứng dụng CARENTO được tự động tái cấu trúc một cách thông minh khi vận hành trên thiết bị di động.

- Do hệ thống kế thừa cơ chế Layout Fluid và cấu trúc khung xương thành phần (Component-driven) đồng bộ từ file cha, quy trình chuyển đổi giao diện trên toàn bộ các trang con (Đăng ký, Đăng nhập, Kho xe, Đơn đặt, Tin nhắn) đều tuân thủ nghiêm ngặt theo hai quy tắc chuyển đổi tự động dưới đây:

4.2.1. Quy tắc chuyển đổi phân hệ Khách hàng (Client Mobile)

- Tối ưu hóa bố cục lưới (Fluid Grid): Tất cả các cấu trúc chia đôi màn hình (như trang Đăng ký/Đăng nhập) sẽ tự động ẩn phần hình ảnh thương hiệu công kênh để tiết kiệm không gian hiển thị, đẩy Form nhập liệu lên chiếm trọn 100% chiều rộng màn hình. Hệ thống lưới của kho xe tự động co từ dạng 3 - 4 cột trên PC về dạng 1 cột duy nhất theo chiều dọc, hình ảnh xe phóng to toàn bề ngang giúp ngón cái của khách hàng dễ dàng vuốt chạm cuộn trang.
- Tối ưu hóa các biểu mẫu nhập liệu (Form Fields): Thanh Navbar tổng đổi sang dạng nút bấm Menu rút gọn (Hamburger Button). Các trường nhập liệu lịch trình và ô nhập Số điện thoại liên hệ được nới rộng khoảng cách đệm (padding), giúp bàn phím ảo của điện thoại kích hoạt chính xác, hỗ trợ người dùng dễ dàng thao tác gõ thông tin bằng một tay khi đang di chuyển.



Hình 2.13: Minh họa cho giao diện đã được hiển thị tốt trên điện thoại phía client

4.2.2. Quy tắc chuyển đổi phân hệ Quản trị viên (Admin Mobile)

- Thu gọn không gian bảng biểu (Responsive Tables): Phân hệ quản trị Admin Panel tự động ẩn thanh Sidebar nền tối vào menu ẩn. Hệ thống các bảng dữ liệu nhiều cột (như trang quản lý khách hàng /admin/customers, trang duyệt đơn đặt xe) tự động kích hoạt thuộc tính cuộn ngang mượt mà (overflow-x-auto) hoặc tự động bẻ dòng cấu trúc từ dạng hàng ngang sang dạng các thẻ khối độc lập (Card layout block) để không gây tràn màn hình.

- Tối ưu hóa không gian Workspace: Tại trang quản lý tin nhắn liên hệ (/admin/messages), phân khu Sidebar danh sách tin nhắn và màn hình hiển thị nội dung lời nhắn tự động chập lại làm một phân tầng điều hướng trước - sau. Khi Admin bấm chọn một tin nhắn, màn hình chi tiết sẽ đẩy ra toàn màn hình, hiển thị Số điện thoại của khách hàng bản lớn sắc nét, hỗ trợ Admin có thể chạm nhanh để sao chép số điện thoại và thực hiện cuộc gọi tư vấn ngoại tuyến lập tức ngay trên điện thoại di động.



Hình 2.14: Minh họa cho giao diện đã được hiển thị tốt trên điện thoại phía admin

CHƯƠNG 3: KIỂM THỬ HỆ THỐNG

1. MỤC TIÊU KIỂM THỬ

Quá trình kiểm thử hệ thống CARENTO được tiến hành trên môi trường Internet thực tế tại địa chỉ tên miền `carento.up.railway.app` nhằm đạt được các mục tiêu cốt lõi sau:

- Đảm bảo tất cả các phân hệ chức năng (đăng ký, xác thực kép, đặt xe, gửi liên hệ, duyệt đơn quản trị) vận hành chính xác 100% theo đúng đặc tả yêu cầu bài toán.
- Kiểm chứng khả năng bắt lỗi và kiểm soát ngoại lệ (Validation) ở cả tầng Frontend (JavaScript) và Backend (Laravel Controller), ngăn chặn tuyệt đối lỗi hệ thống (văng lỗi 500).
- Đảm bảo an toàn an ninh mạng: Rà soát cơ chế bảo mật phân quyền Middleware, kiểm tra tính toàn vẹn dữ liệu khi có biến động giao dịch dữ liệu chéo giữa các bảng trong MySQL.

2. PHƯƠNG PHÁP KIỂM THỬ

Đồ án áp dụng phương pháp Kiểm thử hộp đen (Black-box Testing). Tiến hành kiểm thử tập trung vào chức năng của hệ thống dưới góc nhìn của Người dùng cuối (Khách hàng) và Nhà quản trị (Admin) thông qua việc truyền tải dữ liệu vào hệ thống và đối chiếu kết quả trả về hiển thị trên giao diện, không can thiệp sâu vào cấu trúc dòng mã nguồn nội bộ khi đang chạy Test.

3. CÁC TRƯỜNG HỢP KIỂM THỬ

3.1. Trường hợp kiểm thử 1: Đăng ký tài khoản thành viên mới (RQ01)

Mã TC	Tên kịch bản kiểm thử	Các bước thực hiện	Dữ liệu đầu vào (Test Data)	Kết quả mong đợi hiển thị	Trạng thái
TC01.1	Đăng ký	1. Truy cập form Đăng ký.	- Name: Quỳnh Anh	Hệ thống tạo bản ghi mới	Đạt

	thành công với dữ liệu hợp lệ.	2. Nhập đầy đủ thông tin sạch. 3. Nhấn nút "Tạo tài khoản".	- Email: quynhanh01172004@gmail.com - Password: 12345678 - Confirm: 12345678	trong bảng users, tự đăng nhập, điều hướng về /dashboard.	
TC01.2	Chặn đăng ký khi trùng Email hệ thống.	1. Nhập lại Email đã dùng ở bước TC01.1. 2. Điền các trường khác hợp lệ. 3. Nhấn "Tạo tài khoản".	- Name: Quỳnh Anh - Email: quynhanh01172004@gmail.com - Password: 12345678 - Confirm: 12345678	Tiến trình bị chặn, xuất thông báo lỗi đỏ: "Email này đã được sử dụng trên hệ thống".	Đạt
TC01.3	Chặn đăng ký khi mật khẩu không trùng khớp.	1. Nhập các trường hợp lệ. 2. Ô xác nhận nhập mật khẩu khác ô chính. 3. Nhấn "Tạo tài khoản".	- Name: Quỳnh Anh - Email: qanh0117@gmail.com - Password: 12345678 - Confirm: 87654321	Hệ thống chặn lại, báo lỗi: "Mật khẩu xác nhận không trùng khớp".	Đạt

Bảng 3.1: Bảng kiểm thử Đăng ký

3.2. Trường hợp kiểm thử 2: Xác thực Đăng nhập (RQ02)

Mã TC	Tên kịch bản kiểm thử	Các bước thực hiện	Dữ liệu đầu vào (Test Data)	Kết quả mong đợi hiển thị
-------	-----------------------	--------------------	-----------------------------	---------------------------

TC02.1	Đăng nhập tài khoản User thành công.	1. Truy cập trang đăng nhập. 2. Nhập Email, Mật khẩu User. 3. Nhấn "Đăng nhập".	- Email: quynhanh01172004@gmail.com - Password: 12345678	Đăng nhập thành công, Middleware nhận diện và điều hướng về /dashboard (Client)
TC02.2	Đăng nhập tài khoản Admin thành công.	1. Nhập Email và Mật khẩu của tài khoản Admin. 2. Nhấn nút "Đăng nhập".	- Email: admin@carento.com - Password: admin123	Đăng nhập thành công, Middleware nhận diện quyền điều hướng thẳng vào /admin/dashboard .
TC02.3	Chặn đăng nhập khi sai thông tin.	1. Nhập đúng Email nhưng gõ sai Mật khẩu. 2. Nhấn nút "Đăng nhập".	- Email: quynhanh01172004@gmail.com - Password: saimatkhau	Chặn truy cập, xuất thông báo cảnh báo: <i>"Thông tin tài khoản hoặc mật khẩu không chính xác"</i> .
TC02.4	Tích hợp Đăng nhập nhanh Google OAuth 2.0.	1. Click nút "Tiếp tục với Google". 2. Chọn tài khoản Gmail cá nhân trên Pop-up Google.	- Tài khoản xác thực Google API: quynhanh01172004@gmail.com	Socialite bóc tách Token, tạo Session đăng nhập an toàn, chuyển hướng mượt mà về trang chủ Client.

Bảng 3.2: Bảng kiểm thử Đăng nhập

3.3. Trường hợp kiểm thử 3: Xem chi tiết thông số phương tiện xe (RQ06)

Mã TC	Tên kịch bản kiểm thử	Các bước thực hiện	Dữ liệu đầu vào (Test Data)	Kết quả mong đợi hiển thị
TC03.1	Hiển thị thuộc tính chi tiết xe hợp lệ.	1. Tại Grid trang chủ, click chọn xe Mazda 3. 2. Kiểm tra thông tin.	- Lệnh click tương tác theo ID xe: id = 3	Điều hướng sang /xe/3, hiển thị nét căng ảnh xe, giá tiền thuê, loại hộp số và nhiên liệu.
TC03.2	Bẫy lỗi ngoại lệ khi nhập sai ID xe trên URL.	1. Cố tình gõ sai mã ID xe không có thật lên thanh URL trình duyệt.	- Địa chỉ URL nhập tay: carento.up.railway.app/xe/9999	Hàm findOrFail bắt ngoại lệ, không sập web, trả về giao diện chuẩn 404 - Phương tiện không tồn tại.

Bảng 3.3: Bảng kiểm thử Xem chi tiết xe

3.4. Trường hợp kiểm thử 4: Đặt đơn thuê xe (RQ07)

Mã TC	Tên kịch bản kiểm thử	Các bước thực hiện	Dữ liệu đầu vào (Test Data)	Kết quả mong đợi hiển thị
TC04.1	Tính tiền và đặt xe thành công với lịch trình chuẩn.	1. Tại trang đặt xe, điền thông tin khẩn cấp. 2. Chọn mốc ngày chuẩn. 3. Nhấn "Xác nhận đặt xe".	- Tên lái: Quỳnh Anh - SĐT: 0912345678 - Nhận: 05/06/2026 - Trả: 07/06/2026 (02	JS tự nhảy tiền nhân với đơn giá xe. Đơn đặt nhảy vào bảng bookings với trạng thái pending.

			ngày)	
TC04.2	Chặn đặt xe khi chọn sai niên độ lịch trình.	1. Chọn ngày trả xe nằm trước ngày nhận xe. 2. Ấn nút "Xác nhận đặt xe".	- Nhận: 05/06/2026 - Trả: 03/06/2026	Hệ thống Backend chặn tiến trình gửi form, xuất lỗi đỏ: " <i>Ngày thuê xe không hợp lệ. Vui lòng kiểm tra lại!</i> ".

Bảng 3.4: Bảng kiểm thử Đặt xe

3.5. Trường hợp kiểm thử 5: Gửi tin nhắn liên hệ hỗ trợ (RQ09)

Mã TC	Tên kịch bản kiểm thử	Các bước thực hiện	Dữ liệu đầu vào (Test Data)	Kết quả mong đợi hiển thị
TC05.1	Gửi thông tin liên hệ hỗ trợ thành công.	1. Vào trang liên hệ /lien-he. 2. Nhập đầy đủ thông tin và lời nhắn. 3. Bấm "Gửi thông tin".	- Khách: Quỳnh Anh - Email: quynhanh01172004@gmail.com - SĐT: 0987654321 - Lời nhắn: "Cần tư vấn thủ tục thuê xe Mazda 3 nhận tại Hà Đông"	Bản ghi lưu vào bảng contact trong CSDL Railway, xuất Toast xanh báo gửi thành công.
TC05.2	Chặn gửi liên hệ khi bỏ trống trường cốt lõi.	1. Điền Họ tên, Email nhưng bỏ trống hoàn toàn ô Số điện thoại. 2. Bấm "Gửi	- Khách: Quỳnh Anh - SĐT: (Để trống) - Lời nhắn: "Cần tư vấn xe"	Bộ lọc Validate chặn lại, xuất lỗi cảnh báo: " <i>Vui lòng nhập số điện thoại để nhân viên</i>

		thông tin".		<i>gọi lại hỗ trợ".</i>
--	--	-------------	--	-------------------------

Bảng 3.5: Bảng kiểm thử liên hệ

3.6. Trường hợp kiểm thử 6: Admin CRUD và Đổi trạng thái xe nhanh (RQ11)

Mã TC	Tên kịch bản kiểm thử	Các bước thực hiện	Dữ liệu đầu vào (Test Data)	Kết quả mong đợi hiển thị
TC06.1	Thay đổi nhanh trạng thái vận hành phương tiện.	1. Vào /admin/cars. 2. Nhấn nút chuyển đổi trạng thái xe Mazda 3 trên hàng.	- Giao thức gửi yêu cầu: PATCH - Route: /admin/cars/3/status	Trạng status trong DB lập tức đảo ngược giá trị (từ available sang maintenance), đổi màu nhãn Grid không reload trang.

Bảng 3.6: Bảng kiểm thử đổi trạng thái xe

3.7. Trường hợp kiểm thử 7: Admin điều phối duyệt đơn và cập nhật doanh thu (RQ13)

Mã TC	Tên kịch bản kiểm thử	Các bước thực hiện	Dữ liệu đầu vào (Test Data)	Kết quả mong đợi hiển thị
TC07.1	Phê duyệt đơn đặt xe, khóa xe đồng bộ hệ thống.	1. Vào mục đơn hàng /admin/bookings. 2. Chọn đơn của khách ở TC04.1. 3. Nhấn "Phê duyệt".	- Hành động: Approve - Đơn hàng ID tương ứng đơn của khách Quỳnh Anh.	Trạng thái đơn chuyển sang approved. Đồng thời, chiếc xe nằm trong đơn tự động chuyển trạng thái sang rentedđể khóa xe.

TC07.2	Kiểm tra tính năng đồng bộ số liệu Dashboard.	1. Quay trở lại trang chủ quản trị /admin/dashboard ngay sau bước TC07.1.	- Hàm thực thi: DashboardController@index	Khối Grid Tổng doanh thu (Total Revenue) tự động cộng lũy tiến thêm số tiền của đơn hàng vừa duyệt ở bước TC07.1.
---------------	---	---	---	---

Bảng 3.7: Bảng kiểm thử xem doanh thu

3.8. Trường hợp kiểm thử 8: Admin tiếp nhận quản lý tin nhắn liên hệ hỗ trợ (RQ14)

Mã TC	Tên kịch bản kiểm thử	Các bước thực hiện	Dữ liệu đầu vào (Test Data)	Kết quả mong đợi hiển thị
TC07.1	Phê duyệt đơn đặt xe, khóa xe đồng bộ hệ thống.	1. Vào mục đơn hàng /admin/bookings. 2. Chọn đơn của khách ở TC04.1. 3. Nhấn "Phê duyệt".	- Hành động: Approve - Đơn hàng ID tương ứng đơn của khách Quỳnh Anh.	Trạng thái đơn chuyển sang approved. Đồng thời, chiếc xe nằm trong đơn tự động chuyển trạng thái sang rented để khóa xe.

Bảng 3.8: Bảng kiểm thử xem các liên lạc

CHƯƠNG 4: KẾT LUẬN

4.1. Kết quả đạt được

Sau quá trình nghiên cứu, thiết kế hệ thống dữ liệu chặt chẽ và lập trình triển khai thực tế, đồ án môn học "Lập trình ứng dụng Web" với đề tài xây dựng hệ thống CARENTO đã hoàn thành trọn vẹn và đạt được các kết quả thực tiễn mang tính ứng dụng cao:

- Về mặt chức năng hệ thống: Xây dựng thành công một chu trình vận hành tự động hóa khép kín. Khách hàng thực hiện tìm xe, xem thông số kỹ thuật chi tiết và gửi đơn đặt xe trực tuyến mượt mà. Phân hệ Admin Dashboard cung cấp công cụ thống kê tài chính chính xác và quản lý kho xe, danh sách thành viên khoa học.
- Về mặt công nghệ và bảo mật tối ưu: * Ứng dụng thành công cấu trúc phân lớp sạch MVC của Laravel v13.9 kết hợp với hệ quản trị MySQL v8.0.
 - o Tích hợp hoàn hảo cơ chế xác thực kép an toàn: Đăng nhập truyền thống bảo mật mã hóa một chiều Bcrypt kết hợp giải pháp đăng nhập nhanh chuẩn công nghiệp Google OAuth 2.0 (Socialite).
 - o Giải quyết dứt điểm các bài toán Front-end thông qua Vite Compiler và Tailwind CSS: Loại bỏ hoàn toàn lỗi rớt dấu phong tiếng Việt bằng cách đồng bộ hóa cấu hình font Inter toàn hệ thống, tối ưu dung lượng assets thông qua việc chia cấu hình riêng biệt (customers.css, messages.css), và bảo mật an toàn mã an ninh nguồn (GOOGLE_CLIENT_SECRET) cách ly khỏi GitHub thông qua tính năng Variables của Cloud Server Railway (carento.up.railway.app).

4.2. Những hạn chế còn tồn tại

Mặc dù hệ thống đã hoạt động ổn định trên môi trường internet thật, tuy nhiên do giới hạn về mặt thời gian nghiên cứu và kinh nghiệm thực tế, đồ án vẫn còn tồn tại một số hạn chế nhất định cần được khắc phục:

- Phân hệ đặt xe và thanh toán mới dừng lại ở mức độ tính toán hóa đơn và xử lý trạng thái giao dịch mô phỏng ở Backend, chưa liên kết dòng tiền thực tế với các cổng thanh toán điện tử trực tuyến tại thị trường Việt Nam.
- Luồng nghiệp vụ liên hệ hỗ trợ đã số hóa khâu tiếp nhận thông tin số điện thoại của khách, tuy nhiên khâu liên lạc đàm phán chốt hợp đồng vẫn phải thực hiện thủ công ngoại tuyến (Offline) qua cuộc gọi thoại của tổng đài viên.
- Cơ sở dữ liệu kho xe chủ yếu là tập dữ liệu mẫu phục vụ demo kiểm thử hộp đen, chưa bao quát hết độ phức tạp về biến động giá thuê xe theo các dịp cao điểm (Lễ, Tết).

4.3. Hướng phát triển trong tương lai

Để phát triển CARENTO trở thành một giải pháp thương mại điện tử đột phá, có khả năng cạnh tranh mạnh mẽ trên thị trường số, các định hướng nghiên cứu và phát triển tiếp theo sẽ tập trung vào:

- Tích hợp cổng thanh toán trực tuyến: Tiến hành nghiên cứu tài liệu kỹ thuật API và liên kết trực tiếp dòng tiền thật của dự án với các cổng thanh toán nội địa uy tín như VNPay, Momo hoặc ZaloPay tại trang thanh toán đặt xe.
- Tích hợp Bản đồ số (Google Maps API): Nhúng trực tiếp bản đồ định vị thông minh vào trang chi tiết phương tiện giúp khách hàng dễ dàng tra cứu tọa độ, khoảng cách di chuyển thực tế từ vị trí cá nhân đến bãi nhận xe của nhà xe CARENTO.
- Tự động hóa kênh tương tác: Tích hợp hệ thống tổng đài ảo VoIP tự động thiết lập cuộc gọi kết nối ngay khi Admin bấm nút xử lý tại mục quản lý tin nhắn, hoặc nghiên cứu mở rộng xây dựng phiên bản ứng dụng di động (Mobile App) đa nền tảng để tối ưu hóa trải nghiệm khách hàng mọi lúc mọi nơi.